Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Integrating Model Context Protocol with Unity for LLM-Driven Scene Manipulation in Virtual Reality

Master's Thesis

Author: Efe Oral

Matrikelnummer: 2965307

Supervisor: Dr. David Obremski

Second Supervisor: Prof. Dr. Carolin Wienrich

Chair of Human Computer Interaction

Group of Psychologie Intelligenter Interaktiver Systeme

Julius-Maximilians-Universität Würzburg

February 22, 2026

Abstract (German)

Die Steuerung von Unity Virtual Reality (VR)-Szenen erfolgt traditionell über benutzerdefinierte Skripte, controllerbasierte Interaktionen oder klassische Desktop-Workflows. Das ständige Wechseln zwischen VR-Headset und Desktop-Workflows stört die Immersion und verringert die Intuitivität bei der Entwicklung für VR. Diese Arbeit stellt ein System vor, das lokale, Large Language Models (LLMs) über das Model Context Protocol (MCP) in die Unity-Engine integriert und so eine sprachbasierte Bearbeitung der Szene ermöglicht, ohne die VR-Umgebung verlassen zu müssen. Gesprochene Befehle werden über eine Speech-to-Text-Konvertierung erfasst, von einem lokal gehosteten LLM (Qwen3:8b) in strukturierte MCP-Tool-Aufrufe übersetzt und ausgeführt, was zu Szenenänderungen in nahezu Echtzeit ($\sim$15 Sekunden) führt. Zu den unterstützten Aktionen zur Szenesteuerung gehören das Erstellen, Löschen, Verschieben, Skalieren, Duplizieren, Umbenennen und Ändern von Objektkomponenten und Objektfarben sowie die Steuerung von Lichtquellen. Kontrollierte Auswertungen unter Verwendung von 30 natürlichen Sprachbefehlen ergaben eine durchschnittliche Erfolgsquote bei der Tool-Nutzung von etwa 91% und eine Latenz bei der Ausführung von etwa 15 Sekunden, was zeigt, dass eine sprachgesteuerte, tool-basierte Szenenmanipulation für die interaktive VR-Szenensteuerung praktikabel ist. Eine wichtige Erkenntnis war, dass größere (>20b) und/oder Modelle mit erweiterten Reasoning-Fähigkeiten keine zuverlässigeren Werkzeugaufrufe erzeugten, wie ein Vergleich mehrerer lokaler LLMs ergab. Im Gegensatz dazu übertrafen kleinere Modelle (<8b) mit abgestimmten internen System-Prompts und Parameterkonfigurationen durchweg die größeren Modelle, was darauf hindeutet, dass das Entwurf der System-Prompts und die Parameterkonfigurationen einen stärkeren Einfluss auf die Zuverlässigkeit der Tool-Nutzung haben als die Modellgröße allein. Der modulare, modellunabhängige Aufbau von MCP ermöglicht das Hinzufügen neuer benutzerdefinierter Tool-Funktionen, ohne das Kernsystem zu verändern. In Kombination mit der austauschbaren LLM-Architektur dieser Arbeit bietet es eine nachvollziehbare Grundlage die Beobachtung und den Vergleich des LLM-Tool-Verhaltens von verschiedenen Modellen.

Der Quellcode ist verfügbar unter: `https://github.com/Efe-Oral/Thesis`

Schlüsselwörter: Virtual Reality, Unity, Model Context Protocol, LLMs

Abstract (English)

Controlling Unity Virtual Reality (VR) scenes is traditionally done with custom scripts, controller-based interactions, or desktop PC-assisted workflows. Constant switching between the VR headset and desktop workflows disrupts immersion and reduces intuitiveness when developing for VR. This thesis presents a system that integrates local tool-using large language models (LLMs) with the Unity game engine via the Model Context Protocol (MCP), enabling speech-driven scene manipulation without exiting the VR environment. Spoken commands are captured via speech-to-text conversion, translated into structured MCP tool calls by a locally hosted LLM (Qwen3:8b) and executed, resulting in scene modifications in near real-time($\sim$15 seconds). Supported scene control actions include creating, deleting, moving, scaling, duplicating, renaming, modifying object components and changing object colors, as well as managing light sources. Controlled evaluations using 30 natural language prompts showed an average tool-use success rate of approximately 91% and an execution latency of approximately 15 seconds, demonstrating that speech-driven, tool-based scene manipulation is feasible for interactive VR scene control. A key finding was that larger (>20b) and/or thinking-enabled models did not produce more reliable tool calls, as revealed by comparative analysis across multiple local LLMs. In contrast, smaller models (<8b) with tuned internal system prompts and parameter configurations consistently outperformed larger models, suggesting that system prompt design and parameter configurations have a stronger effect on tool-use reliability than model size alone. The modular, model-independent nature of MCP allows adding new custom tool actions without modifying the core system. Combined with the interchangeable LLM architecture of this work, it provides a transparent framework for observing and comparing LLM tool-use behavior across models.

The source code is available at: `https://github.com/Efe-Oral/Thesis`

Keywords: Virtual Reality, Unity, Model Context Protocol, LLMs

Acknowledgments

I would like to thank my family and friends who supported me through this process. Without their kind encouragements, this work would be way harder to complete. I also want to thank my supervisor Dr. Obremski for his advices and feedbacks throughout this work. Lastly, I want to thank the open source community for their comments. The messages and reviews from anonymous reviewers helped me improve clarity of this work.

Contents

Acronyms

AI Artificial Intelligence.

IDE Integrated Development Environment.

LLM Large Language Model.

MCP Model Context Protocol.

PC Personal Computer.

RAG Retrieval-Augmented Generation.

STT Speech-to-Text.

UI User Interface.

VR Virtual Reality.

List of Figures

List of Tables

1. Introduction

Recent years have seen rapid improvements of three powerful technological domains: *Large Language Models (LLMs)*, *Unity game engine* [13], and *Virtual Reality (VR)* [14], as shown by the recent studies and surveys. Each of these domains have developed significantly on their own, yet their intersection remains a largely unexplored area. Unity, in particular, serves as a flexible middle ground for unifying these technologies because of its real-time 3D rendering capabilities, built-in tools, 3rd party packages, cross-platform support, custom scripting in C# and great community. Its modular design and wide range of libraries make it ideal for developing interactive systems that can dynamically manipulate digital environments in VR. And now, the integration of natural language interfaces powered by LLMs with the **Model Context Protocol (MCP)**, adds a significant new layer of intelligence and flexibility to Unity's already very powerful ecosystem. Integration of LLMs not only increases flexibility for developers and researchers, but also offers a powerful framework for scene manipulation in VR. This further enhances Unity's position as an overall versatile program for applications and research exploration.

In recent years, AI systems have been used in many modalities like: text, image, video, audio etc. and the trends suggest that they will keep growing and evolving [15]. However, it has been a challenge to bring these systems together with interactive **3D environments**. LLMs, while highly capable in text reasoning and semantic understanding, often lacks the structured interfaces necessary to control dynamic graphical environments [16] like in Unity. Some of the main challenges are, **the absence of standardized communication protocols**, **limited domain-specific training data** and **relative niche status of interactive 3D worlds amongst other modalities** such as plain-text. These challenges have constrained the development of tool-using AI agents within Unity and other similar 3D environment settings.

The emergence of the MCP protocol by Anthropic introduces a promising solution to this standardized communication limitation. MCP provides a structured interface for an LLM to discover, describe, and execute **actions** on external systems. The official MCP website describes the protocol as: *"Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems"*[17].

In MCP architecture, shown in fig. 1 *"With MCP"*, the MCP Host is an AI application such as Claude Desktop or Cursor IDE. Through this thesis, Unity also functions as an MCP host, but it is not technically a traditional MCP host because it does not contain the LLM directly in it. However, it is extended with MCP host-like capabilities by serving as the user-facing entry point and coordination with the MCP client (via the **bridge** component discussed in section 4.1) to drive tool invocation.

MCP host contains the MCP client that connects to an MCP server. The server exposes specific functionalities as callable MCP **tools**, which the client can invoke when needed. Each MCP tool corresponds to a well-structured action to be executed in the external application such as in Unity (e.g., creating, modifying, or deleting objects), and can be invoked through natural language. The LLM embedded in the MCP host, acting as an autonomous **agent**, interprets the user's intent based on the prompt and generates an appropriate tool call. The MCP client then forwards this tool call to the MCP server, which executes the tool and returns the execution result to the host [1].

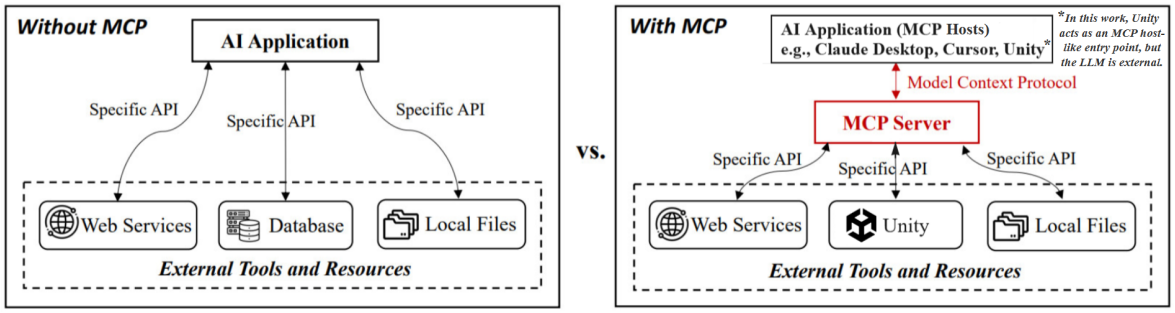


Figure 1: Tool invoking with and without MCP architecture (based from [1], p.3).

Comparison between traditional tool invocation and the MCP-based approach can be seen in fig. 1. In a conventional setup (*left*), an AI application accesses external resources, such as web services, databases, or local files, through **individually** defined APIs. Under the MCP framework (*right*), the same application functions as an MCP host that communicates with an MCP server via the MCP protocol, and offers a unified and consistent interface for interacting with external tools. Eliminating the need for defining individual APIs for each application.

Through this framework, LLMs evolve from passive text generators into **tool calling interactive agents** capable of reasoning about context and directly manipulating the 3D environment inside Unity scene. Integrating MCP enables Unity to communicate

with various LLMs* through a unified, model-independent protocol, eliminating the need for manual API connection for each application, as shown in fig. 1. MCP also allows for **horizontal scalability** [18]. Once an MCP server is configured, additional MCP tools and even additional MCP servers can be seamlessly introduced to expand the system's capabilities without modifying the core communication pipeline, more on this in section 3.4. This architecture lays the groundwork for consistent, near real-time interaction between user, LLM, and 3D spaces, boosting the development of intelligent systems within VR.

Even though the MCP protocol emerged relatively recently, in November 2024 [19], it has already seen rapid adoption within the language model ecosystem. Leading AI developers, including Anthropic, Google [20], Microsoft [21] and OpenAI [22], have integrated MCP into their systems as a standardized mechanism for tool usage in agent-based architectures. This rapid adoption shows that MCP could likely be the industry standard for connecting LLMs with external systems. Similarly, Unity also stands as an industry-standard application for creating interactive 3D content and doing scientific research. And continuously expanding its role in VR and simulation research. Studying the interaction between Unity and MCP at this early stage provides both novelty and foresight to this thesis. Such preliminary exploration can guide how near real-time, LLM-driven virtual environments evolve and become standardized in the future years. Therefore, this thesis explores the research question:

RQ: *How can LLMs be integrated with MCP protocol and Unity to enable developers to manipulate and control VR scenes directly through natural speech, without leaving the VR environment?*

To address this question, this work contributes the following:

- Unity is extended with MCP host-like functionality via a bridge component, enabling communication with local LLMs and tool invocation through the MCP protocol.
- Integration of open-source local Ollama LLMs into Unity, enabling them to interact with the MCP server and invoke tools directly through the Unity interface.
- Adding new Unity-specific MCP tool actions for scene manipulation in VR.

*In order to invoke MCP tools, the chosen language model must support tool calling

- Implementing Speech-to-Text **(STT)** input modality for near real-time voice-based interaction while staying in the virtual environment.

We will look at the design choices, implementations, and the methodology to achieve these contributions in detail in the following sections. However before proceeding, the review of the primary motivations which lead to these contributions must be addressed as well. The main motivation of this thesis is to connect LLMs with interactive 3D environments in a simple and open way. This is addressed by integrating local open-source LLMs into Unity via MCP protocol, the system enables users to control VR scenes using speech while maintaining data privacy and avoiding dependence on commercial APIs. Its modular structure enables for easy customization, scaling and compatibility with different LLMs. This approach helps move VR toward a more intelligent, flexible, and user-friendly medium for creating and exploring ideas.

2. Background and Related Work

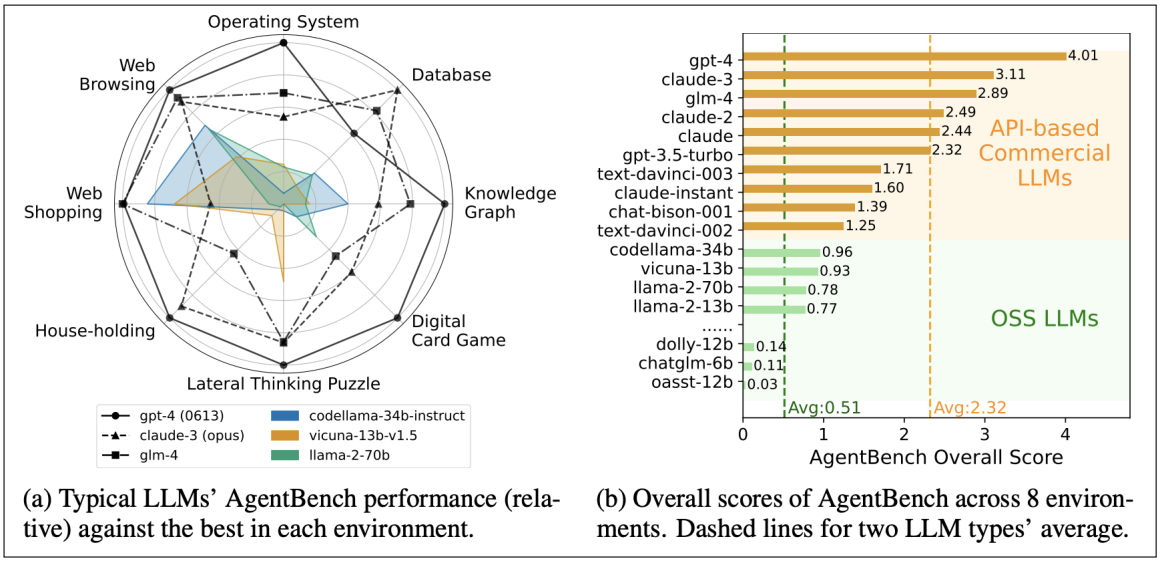
This section provides the necessary background information and summarizes of related works. First, it introduces key technologies such as LLMs and tool-using LLM agents, the relationship between Unity and VR, and the architecture of the model context protocol. Then, the section reviews previous approaches to achieve intelligent systems in Unity. Finally, the section highlights how this thesis differs from the related works, and its novelty.

2.1. Large Language Models

LLMs are deep neural networks trained on massive text datasets. At their core, their goal is to predict the next token in a sequence based on previous ones. They learn about language patterns, structure and meaning through this statistical process. When the user enters a prompt, the model generates text by repeatedly predicting the next most likely token will be until the response is complete [23] [24]. LLMs have become widely used across many domains, because of their powerful nature. A recent survey [25] shows various application domains including mathematics, physics, chemistry, biology, the humanities, and social sciences. Some of their main use cases are summarizing scientific literature, code generation, reviewing legal documents and conversational agents.

ChatGPT, Claude and Gemini are some of the most popular LLMs, according to website traffic data from September 2025 [26] and global usage research on generative AI [27]. These commercial systems show strong performance in reasoning, structured output generation, and tool-related decision-making. Their strength largely results from their large parameter counts, extensive training datasets, and refined reinforcement-learning processes. In comparison, open-source local LLMs have made significant progress in tool reasoning but still show lower reliability and consistency in complex, multistep tasks. The study *AgentBench: Evaluating LLMs as Agents* [2] points out this performance gap, showing that commercial API-based models outperform most open-source counterparts in simulated, tool focused reasoning tasks. The study evaluates a wide range of models, including GPT-4, Claude-3, Llama-2:70b, and CodeLlama:34b, across various agentic domains including web browsing, knowledge-graph reasoning, and task planning. As a reminder, **agents** refer to LLMs capable of using external

tools to perform goal-oriented actions. fig. 2 indicates that commercial models, especially GPT-4 and Claude-3, beat open-source models like Llama-2 and Codellama in almost every category. The performance gap is mostly caused by larger model sizes, better training data, and advanced reinforcement learning approaches that increase tool use reasoning reliability.



(a) Typical LLMs' AgentBench performance (relative) against the best in each environment.

(b) Overall scores of AgentBench across 8 environments. Dashed lines for two LLM types' average.

Figure 2: Comparison of commercial and open-source LLMs on agentic performance across multiple task domains, (adapted from [2], p.1).

However, even though commercial models outperform open-source models in agentic tasks, such huge models require costly computations and strictly dependent on cloud-hosted infrastructures to operate. This makes it really difficult to use them locally on a regular home computer or on a remote university server. Which for this thesis is not feasible because one of the key motivation is to achieve local hosted LLM support. The motivation for this choice can be found in more detail under the Motivation section 3. But in short, running LLMs locally ensures data privacy, long-term reliability, and model-independent design. Therefore, this work's focus towards smaller ($<$ 8b), open-source tool-using models that can operate efficiently on local hardware.

Luckily, even the smaller LLMs can perform strongly under the right conditions. LLMs can be *modified* or *adjusted* through various techniques such as calibrating hyperparameters (e.g., temperature and top_k), changing the system prompt, and through fine-tuning and quantization. Recent studies demonstrate that such modification techniques improve the performance and behavior of LLMs [28]. Hyperparameters calibration in particular allows for quick and easy custom behaviour of the LLM. The key

hyperparameters modified in this thesis and their definitions are summarized in **Table 1**.

This modification flexibility allows developers to tailor LLMs to specific tasks or hardware limits. For these use cases, applications like **Ollama*** is suitable. Ollama is a platform, hosting hundreds of various types of open-source LLMs. Ollama allows users to run various open-source LLMs locally, such as Llama 3, Gemma, and the primary model of this thesis, a **Qwen3:8b-based model**[29] which is derived from Qwen3:8b but adapted with a Claude-style system prompt and adjusted inference parameters. For simplicity, the remainder of this thesis refers to Qwen3:8b-based model as **Qwen3:8b**, unless stated otherwise. The models hosted on Ollama differ in size, performance, and specialization, making them suitable for different tasks such as tool calling which is a requirement when using the MCP protocol. While trying to determine the primary model for this thesis, more than 35 LLMs were tested, complete list of tested models can be found under Appendix B. Finally Qwen3:8b was selected for its speed, tool-call generation reliability, and overall performance. Detailed comparisons between different models and the reason of choosing Qwen3:8b can be found under **Evaluation** section 5.2.

Hyperparameter	**Definition**
streaming	Enables the model to send output tokens incrementally as they are generated
thinking	Controls whether the model performs extended reasoning steps before producing its output, disabling it reduces latency
temperature	Controls randomness in token selection. Lower values produce more deterministic answers, higher values increase creativity
top_k	Limits the number of most likely tokens considered at each generation step, which controls output variety
system prompt	A predefined instruction that sets the model's role and behavior, guiding how it interprets and responds to user input

Table 1: Modified hyperparameters in this thesis and their definitions [12].

As mentioned before, even the small (under ∼8b) LLMs can be highly effective.

*https://docs.ollama.com/

Even more effective once they are configured [30]. For example, Qwen3:8b by default comes with `thinking:true` mode. Through the tests, it has been concluded that the thinking mode does not make a significant difference for tool calling, in fact it was even slowing down the tool calling process by about three times because of the model's additional thinking time. Therefore, the thinking feature was disabled by adjusting the corresponding hyperparameter, which can be easily configured, i.e., `thinking:false`. These types of configurations can be made for specific use cases like for this thesis, where the main objective is fast tool calling with MCP protocol and do not need thinking capabilities of Qwen3.

LLMs are great at and mainly used to generate free-form, human-readable plain text. But in the case of the MCP protocol, it requires a **structured** JSON format for communication, see the example in fig. 3.

(a) Unstructured output

```
"The cube has been successfully
    created in the scene."
```

(b) Structured JSON output

```
{
  "jsonrpc": "2.0",
  "method": "create_object",
  "params": {
    "type": "cube",
    "position": [0, 0, 4]
  },
  "id": 1
}
```

Figure 3: Comparison between unstructured, plain-text (a) and structured output (b).

In order to better structure the JSON output, tool-using LLMs are used. When it comes to creating structured text, tool-using LLMs are just more consistent, dependable, and schema-aware. This is because they are fine-tuned and set to produce structured formats with greater consistency [31]. They have a specialized dataset from which they learn to stick to strict schemas, handle nested fields, and prevent syntax errors. This allows tool-using LLMs to produce better structured, accurate output forms, such as JSON. **JSON-RPC 2.0** is the underlying message format of the MCP protocol [32]. Through this format, the LLM can express a tool invocation by filling in the required and optional fields that correspond to the tool's parameters. Each tool

has its own required and optional parameter fields to be filled. Figure 4 shows an example MCP tool, `manage_editor`, which exposes the Unity editor's state and settings through defined parameters such as `action`, `wait_for_completion`, and `layer_name`. It shows how tools declare required and optional arguments, expected return values, and example usage instructions within the tool schema.

`manage_editor` Tool Schema

Controls and queries the Unity editor's state and settings.

```
Args:
    action: Operation (e.g., 'play', 'pause', 'get_state', 'set_active_tool', 'add_tag').
    wait_for_completion: Optional. If True, waits for certain actions.
    Action-specific arguments (e.g., tool_name, tag_name, layer_name).

Returns:
    Dictionary with operation results ('success', 'message', 'data').
```

Parameters:

- **action**: No description
- **wait_for_completion**: No description
- **tool_name**: No description
- **tag_name**: No description
- **layer_name**: No description

Figure 4: Example MCP tool `manage_editor`.

The LLM analyzes the user's prompt, generates a structured JSON message, fills the tool parameters which is then passed to the MCP client. The client then sends the message the MCP server to execute the requested functionality. This standarized interaction schema combined with natural language input allows scene operations to take place in the Unity VR environment.

2.2. A Brief History of LLM Agents

This section provides the background information needed to understand how LLMs have changed over time from early text-based systems to today's state-of-the-art, tool-using LLM agents. The timeline structure of this section is inspired by UC Berkeley's lecture materials [33] on the progression of LLMs.

LLMs were initially used for language understanding and generation. Their role was limited to producing plain-text based on statistical patterns, as talked about in

section 2.1. This passive behavior made them less suitable in dynamic environments. Early LLMs could only generate replies based on their training data. They were unable to access or reason about additional information. In a world of constantly changing knowledge, their dependency on pre-learned dataset makes them unsuitable for many real world tasks. An intermediate step toward getting over this problem was Retrieval-Augmented Generation (RAG). It allowed models to retrieve relevant **external data** and incorporate it into their context window before generating a response [34]. While RAG addressed the issue of outdated knowledge, it remained unable to perform actions with external systems. To overcome this, research has focused toward agentic LLMs, which are models capable of reasoning and can use tools such as web search or APIs. This transition resulted in a shift from only text-generating models to goal-oriented models capable of using tools and communicating with external systems. Let us review the evolution stages of LLMs, beginning with their early phases and progressing to state-of-the-art tool-using LLM agents.

Early LLMs

Early LLMs such as GPT-2, and LLaMA were designed for text prediction. Their outputs depended entirely on the statistical relationships between tokens learned during pre-training. These models could provide logical and context-aware replies but they lacked the mechanism for interacting with external systems. They operated as standalone text generators, with no access to external tools, memory, or data sources. Once a response was produced, the interaction ended, as illustrated in fig. 5. This basic interaction loop made early LLMs fundamentally passive. They could explain how to complete a task but could not do it themselves. Even though they had high reasoning skills, they were limited to static knowledge and were unable to check, update, or act upon real-world data. As new architectures emerged, these constraints were addressed step by step. One of the most significant advancement was RAG, which allowed LLMs to dynamically retrieve **external information**, expanding their effective knowledge. This advancement was a big step forward from early language models to the agentic systems that followed.

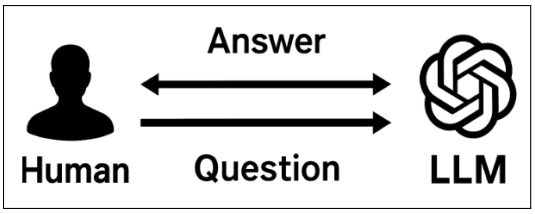


Figure 5: Early LLMs interaction loop. Models entirely rely on text-based question-answer interaction (adapted from [3]).

RAG-Enhanced LLMs

Retrieval-Augmented Generation extended the capabilities of early language models by combining text generation with **real-time information retrieval**. Rather than depending only on the model's training dataset, the RAG system uses the user's prompt to retrieve relevant external information from the web, documentations or a knowledge base. The retrieved information are then fed back into the model's context window, allowing it to generate answers based on up-to-date or domain-specific information. This retrieval process effectively expanded the usable knowledge of LLMs without the need for retraining the model. This mitigated the problem of outdated or incomplete data that limited the earlier models.

As illustrated in fig. 6, the RAG-enhanced interaction loop introduces a retrieval stage between the user prompt and the model's response. Before producing its final output, the model searches and obtains more information from an external knowledge source. The RAG approach considerably improved the factual accuracy and flexibility by merging reasoning with information retrieval. However, its scope was to mainly provide knowledge to the model, not to act. The model could retrieve new information and use it, but could not act upon it or modify external systems. These limitations motivated the development of agentic LLMs, which extend reasoning with goal-oriented behavior and tool-assisted executions.

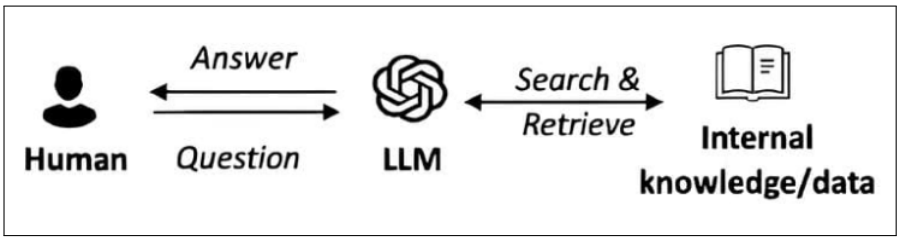


Figure 6: RAG-enhanced LLMs interaction loop. The model retrieves external information before generating an answer (adapted from [3]).

Agentic LLMs

The state-of-the-art step in the evolution of language models is the development of agentic LLMs, models capable not only text generation but also of reasoning, planning, and executing actions using external tools [35]. Unlike RAG systems, which expanded the model's accessible knowledge, agentic models extended their ability to act in the real-world and in external systems. These models can take goals expressed in natural language, translated them into actionable steps, and perform those steps using predefined tools or APIs. As shown in fig. 7, agentic LLMs operate within a broader ecosystem where they interpret user goals, plan actions, and interact with external tools or APIs to achieve those goals.

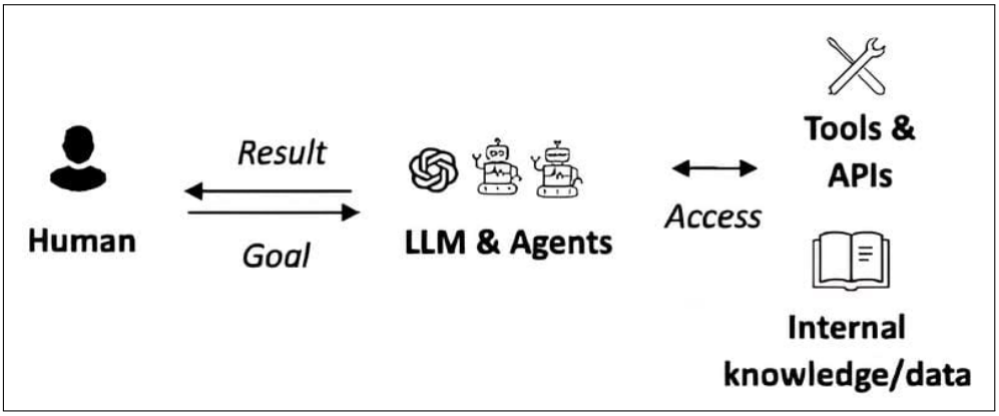


Figure 7: Agentic LLMs interaction loop. Models translate user goals into actions via tools and external resources. (adapted from [3]).

Several key studies established the groundwork of tool-using and agentic LLM behavior. Toolformer [36] demonstrated that with just a few examples, a language model can learn when and how to invoke external APIs through self-supervised fine-tuning, without explicit human labeling. It showed that LLMs can integrate tool usage within their normal text generation process, improving factual accuracy. Similarly, TALM (Tool-Augmented Language Models)[4] introduced the concept of directly integrating tool usage into the model's workflow (see fig. 8). Instead of relying entirely on text generation, the model can **dynamically decide** on the fly to use external tools whenever needed, such as performing a lookup or doing computations, while producing its response. It does this through detection of specific tool tokens. Once these tokens are detected, the model pauses generation, invokes the tool API and then continues writing after receiving the result. As shown in fig. 8, TALM introduces an intermediate step

between input and output, where the model can call external tools and add the results into its response. TALM was different from later systems in that it only used plain-text to handle tool interactions instead of more reliable structured schemas like JSON. This plain-text approach was the initial step toward tool-supported output generation. It provided the groundwork for later structured frameworks like the MCP protocol, and ReAct, which formalized tool communication for better accuracy and reliability.

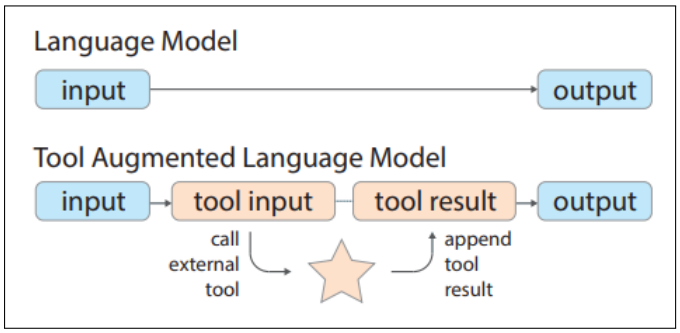


Figure 8: TALM workflow. The model pauses generation to call tool, receives the result in plain text, and integrates it into the final output (adapted from [4]).

TALM and Toolformer addressed the issue of how to call a tool, the next phase of research focused on how to plan, execute, and adapt tool use for multistep complex tasks. ReAct (Reason + Act) [37] introduced a major conceptual shift in the design of tool-using LLMs by explicitly combining *reasoning* and *acting* within a single framework. Earlier systems like TALM and Toolformer could invoke tools but did so primitively, without reasoning but focusing on catching tool tokens. ReAct added a reasoning path, an internal thought process that allows the LLM to explain its reasoning steps before taking an action. This reasoning is then followed by a concrete tool call, whose result is observed and included into the next reasoning step. The loop of **"reason→act→observe→reason again"** allows LLMs to participate in multi-step planning and self-correction, shifting their role from static text generation toward interactive task execution. By allowing the model to maintain an explicit reasoning chain alongside with external actions, ReAct successfully connected the gap between text generation and goal-driven behavior, laying the groundwork for modern agent frameworks.

Later, Voyager [38] came and applied these principles to an embodied 3D environment, making the study one of the first concrete realizations of a fully autonomous, continuously learning LLM agent operating inside a 3D environment. It brought to-

gether the earlier advancements of observation, reasoning, and memory into a single embodied system. Built on top of video game Minecraft, Voyager enabled the model to observe its surroundings, plan actions, execute them through APIs, and iteratively improve by storing successful strategies in a long-term skill library. Each new goal in the game triggered a "reasoning→action→observation" loop similar to that introduced in ReAct[37], but grounded within a spatial, interactive world rather than text-based tasks. This 3D embodiment made the results of language model's reasoning directly observable in the game. Every decision, success, or failure displayed as visible changes in the virtual environment. Such visibility made Voyager a key milestone in evaluating how effectively an agentic LLM can transform high-level textual goals into real, measurable outcomes. Among the studies discussed, Voyager is most closely related to the work presented in this thesis. Both systems explore the integration of LLMs with interactive 3D environments to enable near real-time, goal-driven behavior. However, while Voyager focuses on autonomous exploration and self-improvement inside a game world, this thesis employs a human-guided design. The Unity-based system described here uses an LLM, connected with MCP protocol to manipulate a 3D VR scene in near real-time based on user prompts. Rather than following independent goals, the system interprets user's intent and executes corresponding actions within the VR environment. This difference shifts the focus from autonomous world learning 3D embodied agent to user-driven scene manipulation assistant, where the LLM acts as an intelligent step between the user and the virtual environment. Table 2 summarizes these foundational studies, highlighting their main contributions, mechanisms, and application domains.

In general, LLMs have progressed from static text generators to interactive systems that can use tools, reason and take actions. TALM and Toolformer were two of the first studies that introduced tool invocation. ReAct and Voyager, on the other hand, combined reasoning, planning, and feedback loop into one framework. With these advancements, the field is shifting towards a common operational model known as the **reason–action–observation** loop. In this loop, the model analyzes the user's intent, picks and performs a suitable tool action, observes the result, and then updates its internal context before proceeding. This repeating technique serves as the foundation for modern LLM agents. It gives them flexibility, adaptability, and the ability to communicate with other systems in a methodical manner using formats such as JSON or tool-call schemas [35].

Study	Year	Contribution	Core Mechanism	Example Domain
TALM[4]	May 2022	Integrates tools into model architecture	Tool invocation tokens	Computation/query tasks
Toolformer[36]	Feb. 2023	Self-supervised API calling	Predicts tool usage during generation	Text-based tasks
ReAct[37]	Mar. 2023	Combines reasoning and action loops	Step by step reasoning with tool calls	Question-answer, interactive tasks
Voyager[38]	Oct. 2023	Embodied LLM agent	Continuous virtual environment interaction	3D/Minecraft

Table 2: Comparison of key studies on tool-using and embodied LLM agents.

The impact of this evolution of LLMs extends beyond text generation, knowledge retrieval or tool-calls. Agentic architectures enable LLMs to serve as intermediaries between humans and digital systems, translating natural language into executable goal-oriented operations. As illustrated in fig. 9, this difference separates the models that can only describe the steps to perform the actions from those capable of actually performing them to achieve a goal. The figure shows that how a non-agentic model provides textual instruction steps to create a Unity gameobject, whereas an agentic model actually executes the command autonomously through `manage_gameobject` tool invocation, producing an actual change in the Unity scene.

The growing number of tool-using LLMs have also highlighted the need for a unified communication standard between LLMs and external applications. Earlier methods used separate API connections, which limited their compatibility and scalability. MCP protocol solves this gap by offering a uniform interface via which LLMs can discover, invoke, and manage tools using structured JSON messages. MCP creates a shared communication layer that connects an LLM's reasoning capabilities to the ability of external systems, allowing tools to interact via a single protocol. In this thesis, MCP acts as a link between the LLM agent and the Unity VR environment, converting natural language user commands into near-real-time scene modifications and demonstrating the practical implementation of agentic behavior in a virtual, interactive environment.

Recent work have started to explore agentic LLMs inside game engines and 3D environments. A previously discussed study, Voyager, already demonstrated an au-

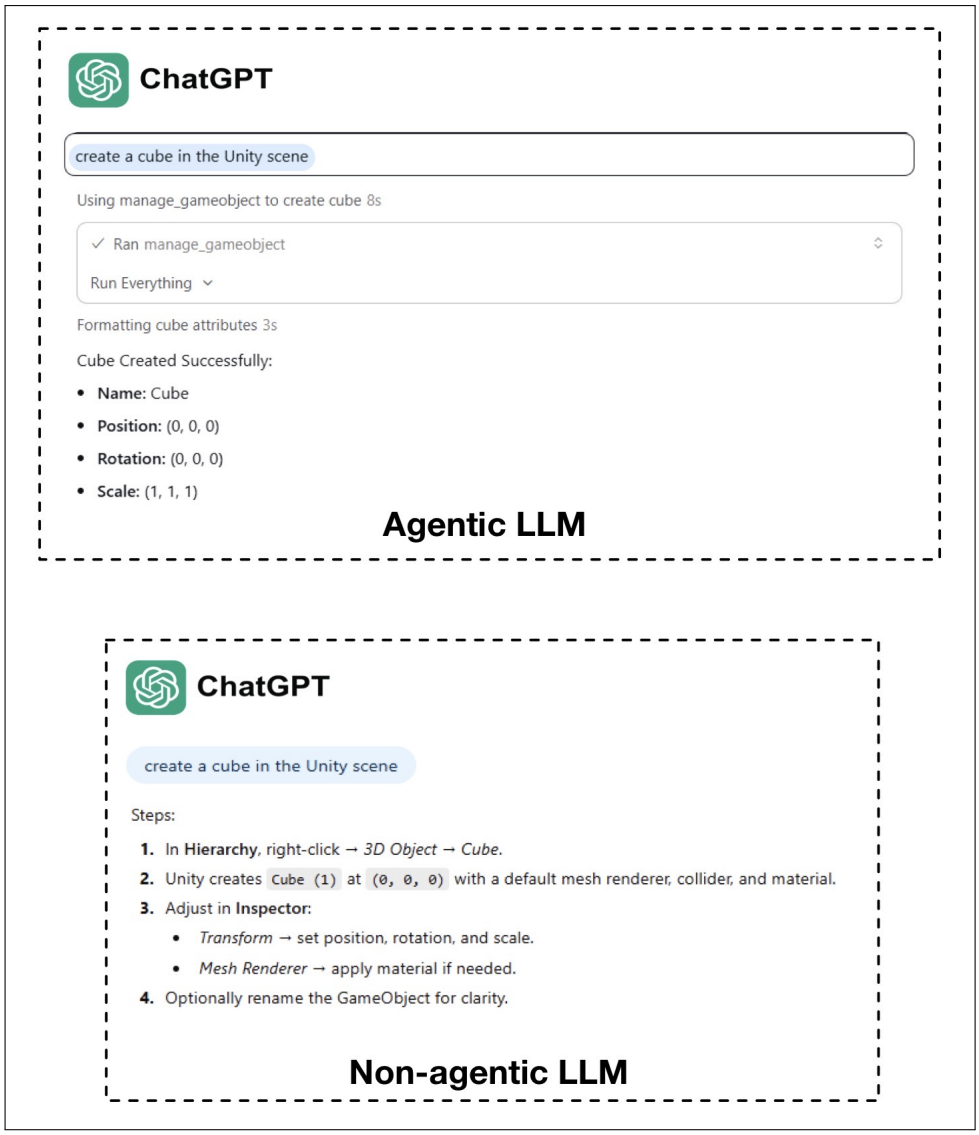


Figure 9: Comparison between agentic and non-agentic LLMs. The agentic model executes the command autonomously through a tool invocation, while the non-agentic model provides step-by-step instructions for user to follow.

tonomous agent that learns skills and completes tasks inside Minecraft, using the game world as an environment for exploration. Other systems such as Unreal Engine integrates LLM based agents for different purposes, such as, environment aware real life scenarios in VR [39], and procedural content generation with multi-agents in Unreal Engine [40]. These projects show that LLM agents can already control and manipulate complex 3D worlds. However, these studies still differ from the scenario focused in this thesis. Existing agents in game engines usually run on computer monitors, focus on autonomous exploration or bulk procedural content generation, and rarely support developer driven precise scene manipulation inside a VR headset. Speech input modality

is not very common and when present, it is often limited to small keyword commands and is not tightly integrated with a structured tool layer like MCP. To author's knowledge, no prior work provides a speech-to-text driven LLM agent that uses MCP tools to help users and developers create and edit Unity VR scenes in near real-time while they stay inside the VR headset. This thesis explores that gap by treating the LLM agent as an assistant for developers: it interprets natural language instructions, invoke MCP tools, and returns concrete changes to the VR scene, turning agentic LLM behavior into a practical aid for interactive VR development.

2.3. Unity and Virtual Reality

Unity is a real-time game engine for interactive 3D content. It targets desktop, mobile, game consoles, web, and VR headsets. Developers use it for games, simulation, training, and research [41]. The engine offers real-time rendering, physics, animation, and multimodal input. These features make Unity a common choice for VR research and prototyping, while creating room for custom improvements.

Unity supports VR devices through its XR system. XR Plugin Management package loads the correct device backend for each headset, which in this thesis Meta Quest 2. This layer hides the device specific details and gives developers a single, unified VR interface. The **XR Interaction Toolkit**[42] builds on top of this layer and provides a ready-made player rig, input actions, locomotion, teleport, floating name tags and grabbing. These templates make VR projects easy to start. This thesis uses the Toolkit as the base and builds on top of it, since the Toolkit already include moving, turning, jumping, and basic interactions. This setup allows VR headsets to work with Unity smoothly and reduced the amount of custom code a developer needs to write. Additionally, Unity still allows to add custom features using C# scripting. The component model makes features modular and easily testable. C# scripting enables quick iteration cycles. Prefabs and ScriptableObjects allow to reuse objects across many experiments. These characteristics make Unity an effective choice for developing and testing interactive systems.

However working with VR also has some practical limitations. A typical problem is that the need to take off the headset, change something on the computer, and then put the headset back on for testing. This procedure is slow, disrupts the workflow, and can be unpleasant during long sessions. These disruptions slow down the iteration process

and affect the overall user experience for the worse. These limitations highlight the need for more direct and hands-free ways of controlling the scene inside VR. This thesis helps address this limitation as well. By using speech-to-text input together with the MCP protocol, scene adjustments can be made directly inside VR without touching external menus or switching back to the desktop.

Overall, Unity provides a common ground for VR runtime, device integration, and quick development workflow. These features makes it a good choice for building an LLM-based scene control system.

2.4. Existing Interaction Paradigms in VR

Virtual Reality supports many ways of interacting with objects in a scene. Common tasks include selecting, moving, adding component, rotating, scaling, duplicating, and creating gameobjects. Today, most VR applications solve these tasks with a mix of menus, raycast-based pointing, controller button mappings, and hand or body gestures. These techniques are well established and have been studied for many years in VR research [43].

A very common method is menu based interaction combined with raycasting. The user holds a controller and points a virtual laser at menu items that appear in front of them or attached to the hand. From fig. 10 can be seen most used menu types, such as linear menus, radial menus, grid menus, and hand attached panels. These menus work, but they can get complex when they have many items or multiple levels. A study [5] reports the following findings about different menu types: raycast-based menus are often slower than more direct techniques, hierarchical menus can be hard to use, and large menus can occupy a significant part of the screen.

Controller buttons are another key part of VR interaction. They are often mapped to operations such as grab, teleport, scale, rotate, duplicate, or open a menu. In practice, each application chooses its own button layout. This is a problem because VR controllers have a limited number of buttons, and there is no single standard mapping for object manipulation tasks. Users must remember different combinations for each application, such as pressing two buttons together, holding a button while moving the joystick, or using long presses for shortcuts. A research [44] on controller interaction design shows that different mappings influence performance, user comfort, and learnability. Results showed that complex mappings increase the mental effort

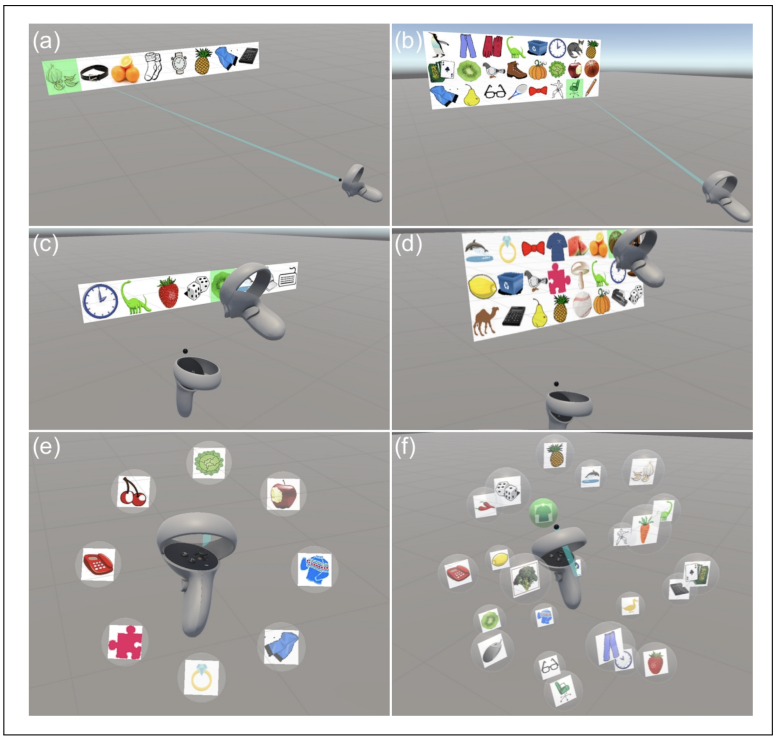


Figure 10: Menu types: raycasting with 8 items in a linear layout(**a**); raycasting with 24 items in a grid layout(**b**); direct input with 8 items in a linear layout(**c**); direct input with 24 items in a grid layout(**d**); marking with 8 items in a 2D radial layout(**e**); and marking with 24 items in a 3D spherical layout(**f**) (adapted from [5]).

needed to remember which combination triggers which command.

These techniques are powerful, but they also have clear limits when used for manipulating or editing scenes inside VR. Creating a new object, duplicating a row of items, or adding components to gameobjects often requires navigating deep menus or learning several button combinations. For many users, this feels overwhelming and not so intuitive. The limited number of buttons on typical VR controllers forces designers to overload inputs with combinations, long presses, and context dependent actions. This leads to different mappings in each application and makes it hard for users to transfer what they learned. Existing VR systems and studies mainly focus on menus, raycasts, controller mappings, and gestures. Speech based interaction appears less common, even it may outperform conventional techniques in some scenarios [45]. And when it appears, it is usually restricted to small sets of fixed keyword commands rather than flexible intelligent scene authoring like in this thesis.

These limitations suggest that current interaction paradigms are well suited for low level object manipulation, but less suited for high level editing and scene manipula-

tion tasks. There is an opportunity to design an interaction approach that preserve the raycast-based direct manipulation for precise adjustments, while reducing the dependency on complex menus and button combinations for more complex operations. This thesis extends existing interaction paradigms by combining speech-to-text input and LLM assisted tool calls via the MCP protocol. The idea is to replace the most complex parts of menu navigation and button combinations with intuitive natural language commands, while still using VR controllers for fine adjustments. Later section 4 describes how this combination works in more detail.

2.5. Model Context Protocol

2.5.1. Motivation and Overview of the MCP Protocol

LLMs work better when they have access to the context about the tasks they are executing. In some systems this is hard to achieve. Common challenges include [46]:

- **Context window limitation:** models can only see a fixed amount of tokens at once, so long background information do not fit the context window.
- **Context fragmentation:** in multi-agent setups, different agents hold different parts of the information, and no single one has the full picture. Though, this is not a problem of this thesis, since this thesis employs only one agent.
- **Context prioritization:** as interactions grow, it becomes difficult to decide which pieces of context are important and which can be dropped.
- **Cross modal integration:** context can come from many sources such as text, structured data, images, or scene state, which are hard to unify and coordinate.
- **Context persistence across sessions:** keeping relevant contextual information across system restarts or multiple interaction sessions requires strong storage and retrieval mechanisms.
- **Context staleness:** in dynamic environments, stored context from trained datasets quickly becomes outdated if it is not refreshed.

The Model Context Protocol was introduced to address these problems. MCP fundamentally follows a client-server architecture and is an open protocol that standardizes

how AI applications and LLMs connect to external tools, data points, and context sources. It defines a common set of messages for discovering tools, calling them, and accessing resources, all using structured JSON messages. Before MCP, model-tool pairs required their own custom API integration, creating a so called *"N*x*M"* problem [47], where N different models must be individually connected with M different tools. This led to rapidly increasing number of isolated connections and leaving models disconnected from live data and external applications. For everyday users, this isolation can be realized as the constant *copy and paste* loop to provide the model with current data and context. To tackle this, MCP provides a single, reusable interface that any client and server can implement. In practice, this means that different agents, tools, and backends can share context and capabilities through one protocol instead of many incompatible ones.

2.5.2. MCP Roles and Architecture

MCP defines how a language model can interact with an external system by answering the questions:

- What tools are available to me?
- What resources or contextual data does the system expose?
- How do I call a tool in a structured and reliable way?

To answer these questions, MCP uses a client–server architecture. This gives a stateful session between client and server and a standard format for all user requests, server responses, and *notifications** [1].

Figure 11 (based on the official MCP architecture diagram [6]) shows the core MCP components: the user talks to the host, the host runs one or more MCP clients, each client connects to one MCP server whether they are local or remote, and the server wraps one or more external systems.

Host: The host is the user facing application, for example Claude Desktop or Cursor IDE. The host contains the LLM, decides which MCP servers are allowed, starts MCP clients, and enforces security and management. In this thesis, Unity is extended to

*MCP notifications let the server tell the client when something changes, such as tool updates, without the client querying. This mechanism eliminates constant polling, increases efficiency, and maintains consistent awareness of the server's functionality.

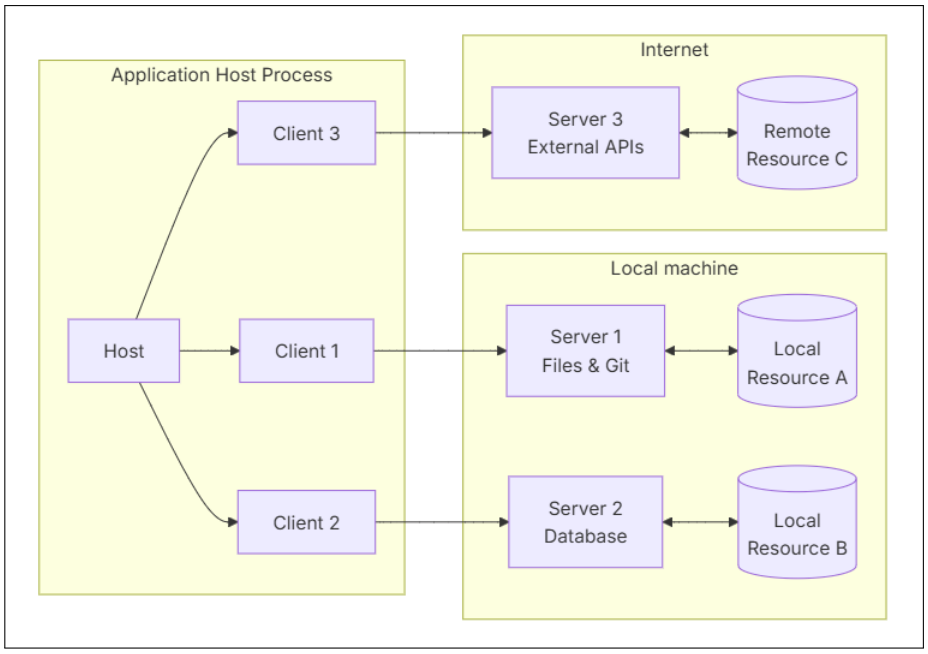


Figure 11: MCP core components (source: [6]).

act as a host-like start point by serving as the user-facing application and coordinating MCP client interactions through a bridge component. While Unity does not contain the LLM itself and therefore is not a traditional MCP host, it achieves a similar role in the interaction pipeline by initiating the interaction.

Client: An MCP client is a protocol component inside the host. It opens a connection to one server over a transport method such as **Stdio** or **HTTP**. Stdio or standard input/output is used for local MCP servers. As stated in the official MCP architecture documentation, *"This* [Stdio] *transport method provides optimal performance with no network overhead"*[48] making it perfect for near real-time applications, hence this thesis prefers this transport method. On the other hand, HTTP transport method enables communication to remote servers. During startup, client and server exchange a handshake that negotiates protocol version and supported features. After that, the client sends JSON-RPC messages in both directions and hides transport details from the rest of the host. Each connection is isolated, so one server cannot see requests intended for another server.

Server: An MCP server is a separate program that exposes resources and tools to clients. A server can run locally (Stdio) or remotely (HTTP). Servers usually sit next to external systems such as file stores, databases, web APIs, or, in this thesis, Unity scene manipulation logic. A server declares its available capabilities and handles

client requests using structured JSON-RPC messages, including requests, responses, and notifications.

MCP consists of two layers, **Data Layer** and **Transport Layer**. Data layer focuses on **what** is communicated between the client and the server. It builds on JSON-RPC protocol and defines the structure, schemas and lifecycle management, including core primitives: tools, resources, prompts, and notifications. The transport layer handles client-server communication channels and authentication. It manages connection format (i.e., Stdio or HTTP), message framing, and secure communication between MCP participants. A session starts with lifecycle management. The client sends an `initialize` request that includes its protocol version and the features it supports. The server replies with its own version and capabilities, such as whether it offers tools, resources, prompts, or notifications. After both sides agree, the client sends a small `initialized` notification and the connection is ready. From this point on, both parties know which features they can safely use.

One of the most important data-layer concept is the set of primitives. MCP defines three main kinds of capabilities that servers can expose: tools, resources, and prompts [46].

- **Tools** represent actions, for example *"run a database query"*, *"send an email"* or *"create a cube in the Unity scene"*. Each tool has a name, return data and JSON schema for its parameters. The client can ask to list tools and then call one by sending a `tools/call` request with structured arguments. An example tool call can be seen in fig. 12.

- **Resources** represent read-only context, such as local files, databases, API responses or any other source that an AI needs to understand context. Once a resource is retrieved, it is then fed to the LLM for its use. It is up to the system to give only the relevant portions or all resource to the LLM. Resources help provide the model with up-to-date information without expanding its context window permanently.

- **Prompts** are predefined templates stored on the server. They help structure tool calls with the language model, often including plain-text few-shot examples or task-specific instructions. Models can use them to get consistent instructions on how to use tools or how to perform common tasks.

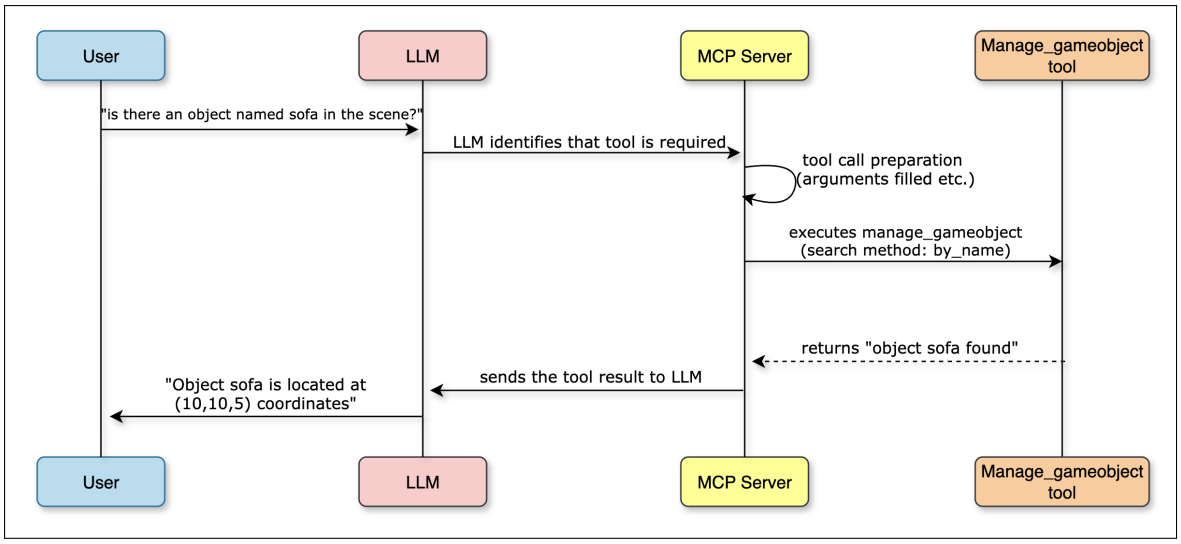


Figure 12: Example tool calling diagram.

Each primitive type has a small set of default methods. For example, servers support discovery via `tools/list`, `resources/list`, and `prompts/list`, and tools can be executed via `tools/call`. This design makes primitives dynamic. A MCP server can change which tools or resources it exposes at runtime, and clients can re-discover them when needed.

Figure 13 gives an overview of end-to-end MCP workflow, highlighting the core components and primitives. A user prompt first enters the MCP host, where the LLM analyses the intent and decides whether tool usage is needed. If so, the host uses its MCP client to send requests over the transfer layer to one MCP server, which exposes tools, resources, and prompts backed by external data sources such as web services, databases, or local files. The server executes the selected tool, returns the result to the client, and the host feeds this result back into the ongoing conversation. This illustrates how host, client, transfer layer, and server work together to turn a user prompt into a concrete action.

Building on this high-level workflow, a typical MCP interaction follows the lifecycle shown in fig. 14. The client and server first run the **Initialization Phase**. The client sends an `initialize request` with its protocol version and capabilities. The server replies with its own version and supported features, then the client sends an `initialized notification`. After this handshake, both sides know which primitives (tools, resources, prompts) they are allowed to use. During the **Operation Phase**, normal work happens. The client submits requests on behalf of the host, such as

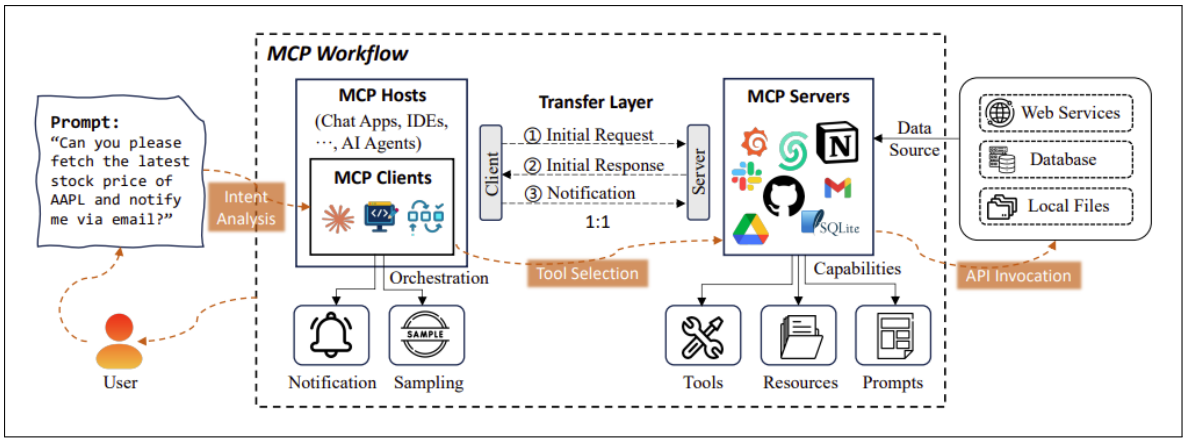


Figure 13: High level workflow of the Model Context Protocol (source: [1]).

`tools/list` to discover available tools, `tools/call` to execute a tool, `resources/list` and `resources/read` to fetch context data, or `prompts/list` and `prompts/get` to retrieve prompt templates and few-shot examples. The server can also initiate requests back to the client, for example `sampling/complete` to ask the host to continue to run the LLM, or `elicitation/create` to ask the host to collect extra input, clarification or confirmation from the user. If both sides support, which is agreed in the initialization phase, they may send **notifications** such as `notifications/tools/list_changed` when the tool set changes, or notify about progress updates for long-running operations. Together, these requests and notifications form the **Normal Protocol Operations** that run in a loop as long as the session is active. Finally, in the shutdown phase, the client tells the server to terminate, the server performs any cleanup it needs, and both sides close the connection.

In this thesis, this lifecycle corresponds to LLM controlling the Unity VR scene. Scene manipulation code is wrapped in an MCP server that exposes tool actions such as creating, moving, or modifying gameobjects. The Unity host sends the user's speech as text to the MCP client (bridge). The bridge adds the tool definitions to the prompt and forwards this payload to local Ollama model. Then the model generates the tool call or calls and forwards them back to the bridge. There it is forwarded to the server for execution. Finally, the MCP server updates the Unity scene and reports the result back to bridge, closing the loop. This concrete use of host, client, server, and primitives is what enables near real-time, tool-driven interaction between the LLM and the VR environment.

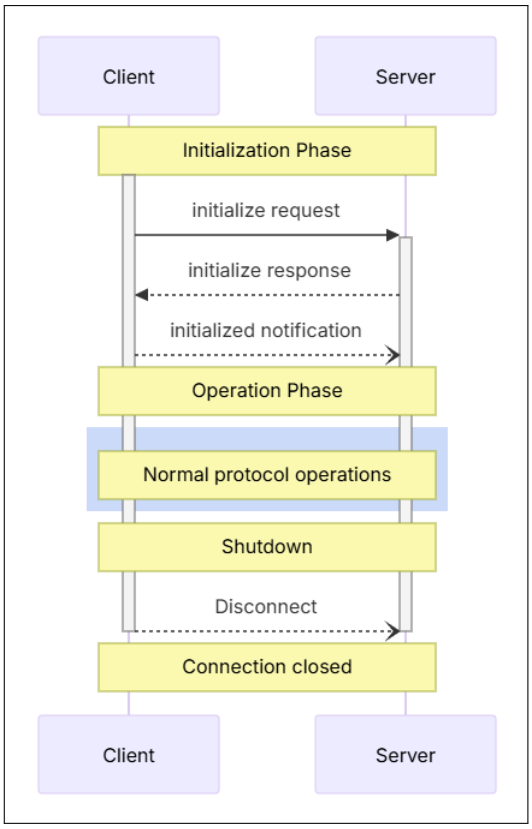


Figure 14: MCP Lifecycle (source: [7]).

2.5.3. Existing MCP-Like Systems

MCP is part of a broader family of protocols that try to give LLM-based systems structured access to tools and even sometimes other agents. A recent survey on agent interoperability [49] compares MCP with three other protocols: Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). MCP focuses on one LLM, talking to available tools and data sources through a JSON-RPC client-server interface. ACP and A2A, in contrast, focus on communication directly between LLM-agents, while ANP targets open, decentralized networks of LLM-agents that can discover and collaborate with each other on the internet. All four protocols aim to improve the reliability and scalability of agentic systems across a wide range of use cases. Although the goals are similar, there are some differences between the four in the scope and design choices.

Table 3 summarizes how these four protocols target different parts of the agent ecosystem. MCP standardizes how an LLM-based application interacts with external tools, resources, and prompts through a structured client–server protocol. ACP reduces communication friction between different LLM agents with an HTTP-based interface.

A2A proposes a common format for agents to call each other, and ANP extends this idea to open, internet based agent collaboration.

Protocol	Primary focus
MCP	Standardized usage of tools, resources, and other context by LLMs.
ACP	Communication and coordination between different agents over a HTTP based interface.
A2A	A common protocol for direct agent-to-agent invocation and compatibility across different agent frameworks.
ANP	Internet oriented agent networks, where agents can discover each other and collaborate over the web.

Table 3: Overview of MCP and related agent communication protocols ACP, A2A, and ANP.

Agent Communication Protocol (ACP)

ACP is an open standard from IBM for agent-to-agent communication and can be realized as collaboration of agents [50]. It uses HTTP based REST messages instead of JSON-RPC and is designed for multi-agent workflows where different agents, each with their own tool sets could collaborate. Messages are sent as multipart HTTP requests that can include text, binary data, and streaming responses in one envelope. ACP has support for multimodal content, asynchronous operations, and routing between many agents. In the IBM overview, MCP is positioned as a protocol that enriches the context of a single model with tools and resources, while ACP sits one layer above it and lets multiple agents talk to each other [51].

Compared to MCP, ACP makes different trade-offs. MCP keeps the protocol simple, relying on a small, well-defined set of primitives (tools, resources, prompts, notifications) and a strict client-server pattern. This is ideal when a single host coordinates an LLM and wants direct, low latency access to many tools, for example in an IDE or in this thesis a Unity VR application. ACP introduces more complexity, but more flexible for coordinating interactions across multiple hosts. It can route messages between many independent agents that may use different internal models or even different protocols such as MCP behind the scenes. In other words, MCP is about how one agent talks to available tools, ACP is about how multiple agents talk to each other. The two

standards are therefore complementary rather than direct competitors [49].

Agent-to-Agent Protocol (A2A)

The A2A protocol is an open standard from Google that gives AI agents a common way to talk to each other. It is designed so that agents built by different vendors and frameworks can exchange messages over standard web technologies such as HTTP, JSON and server-sent events [52]. A2A introduces the idea of an "Agent Card". The Agent Card is a JSON document that functions as a digital business card for the initial discovery and interaction phase. It gives important metadata about an agent. Clients use this data to determine whether an agent is appropriate for a given task, how to structure requests, and how to interact properly.

Key information of agent cards includes identitiy, service endpoint (URL), A2A capabilities, authentication requirements, and a skill list [53]. Figure 15 shows an A2A Agent Card for a simple `"Hello World Agent"`. The card defines the agent's identity (name, version, endpoint), supported input and output modes, capabilities such as streaming, the skills it exposes to other agents, and availability of an extended authenticated card. This organised description allows other agents to discover and use it without having to understand its internal implementation. In practice, A2A focuses on multi-agent workflows in company settings, where many specialized agents must collaborate on long-running tasks [54]. It complements MCP protocol by focusing on communication between agents themselves, whereas MCP focuses on providing tools and context to a single LLM.

```
# This will be the public-facing agent card
public_agent_card = AgentCard(
    name='Hello World Agent',
    description='Just a hello world agent',
    url='http://localhost:9999/',
    version='1.0.0',
    default_input_modes=['text'],
    default_output_modes=['text'],
    capabilities=AgentCapabilities(streaming=True),
    skills=[skill],  # Only the basic skill for the public card
    supports_authenticated_extended_card=True,
)
```

Figure 15: Example A2A Agent Card describing a simple text-based `"Hello World"` agent (source: [8]).

Agent Network Protocol (ANP)

ANP targets a broader, internet scale setting. It is an open communication protocol that aims to be *"The HTTP of the Agentic Web"*, enabling agents to discover each other, authenticate securely, and exchange messages directly over the internet [55]. ANP uses a three layer architecture, as can be seen in fig. 16. At the bottom is the identity and encrypted communication layer, which handles agent identity and secure channels. It is based on W3C DID, so any two agents can verify each other and set up end-to-end secure communication without a central authority. Above that is the meta protocol layer, which decides how two agents will actually talk. In this layer, agents negotiate protocol details such as message formats, interface calling methods, and session management detials, often driven by natural language descriptions of their requirements and capabilities. And at the top the application protocol layer, which focuses on what the agents can do. Here, agents publish structured descriptions of their functions and interfaces and offer discovery mechanisms so that other agents can find them and call the correct entry points. Overall, these three layers let ANP provide secure, flexible communication negotiation, and web-style agent description and discovery [9]. In this way, agents can publish structured descriptions of their capabilities, discover other agents, and build peer-to-peer connections without going through a central platform. ANP focuses on large scale, open agent networks. Whereas MCP protocol is used with a single host and standardizes how an LLM access to tools and resources. In short, MCP is used to integrate tools under one agent, while ANP defines how multiple agents connect and collaborate across the wider internet.

In summary, MCP, ACP, A2A and ANP all address gaps in how LLM-agent systems interact, but with different focus. MCP standardizes how tools, resources, and prompts are exposed to a LLM through a lightweight JSON-RPC client–server protocol. ACP focuses on HTTP based communication between different agents, giving them a REST style interface for exchanging richer messages. A2A defines a standard web-friendly format, via Agent Cards, so that agents can describe their skills and call each other in a standardized way. ANP goes one step further toward an "agentic web" by adding identity, secure channels and decentralized discovery, so agents can form open networks across the internet. As a reminder, summary of the protocols can be found here table 3. This thesis stays within MCP's more constrained scope, where one LLM needs reliable, low latency access to a well-defined set of tools. MCP is therefore the best fit,

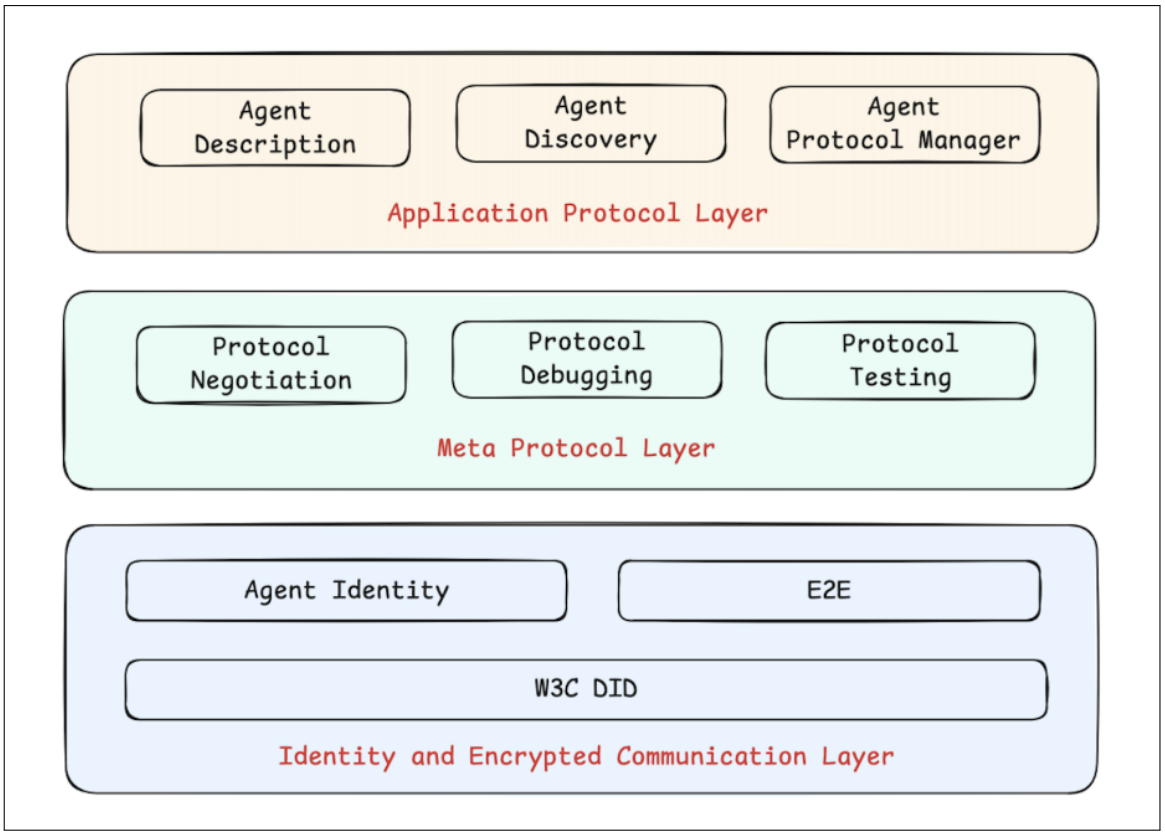


Figure 16: Three layer architecture of the Agent Network Protocol (source: [9]).

because it lets the Unity and the MCP server remain independent components, while still exposing Unity VR scene operations to the LLM through a clear, model-agnostic interface.

Related Work

Before talking about related works, it is necessary to first define what counts as related. In this thesis, related works counts as systems that manipulate or control 3D virtual environments, whether in VR or in computer screens, with any type of modality (text-based, speech-based or some different forms) using LLMs. In this section, related works with a similar goal but with different approaches will be discussed. They will be revived in terms of their scope, input modality, interaction platform, and LLM deployment configuration (local or commercial). Also their similarities/differences, design choice approaches, strong sides, and limitations. Although the interaction with LLMs and virtual scenes are relatively understudied [16], there are promising studies with good results. A recent survey [56] on 3D LLMs summarizes the recent trends and shows that LLMs are used for 3D understanding, scene manipulation, 3D task planning, and mixed reality applications. Building on this broader picture, this section focuses on four studies: **Unity's own AI Assistant** [57], which is integrated into the Unity 6.2 editor and can control 3D scene as well as can generate C# code, and game assets from natural language text prompts. **LLMR** [10], a study exploring the usage of LLMs to create and edit interactive mixed reality scenes in Unity. **Chat-Edit-3D** [11], a dialogue-based interactive system for editing 3D scenes with text prompts. And **AssistVR** [58], a Unity-based VR system that combines speech commands with raycast selection for LLM assisted object manipulation.

Unity AI Assistance

Unity AI assistant is a set of AI tools built into the Unity 6.2 editor [57]. It provides a chat interface window in the Unity editor that accepts natural language text commands. From this window, developers can describe scene changes and the Unity AI assistant create or modify the scene based on the prompt. Capabilities of the AI assistant include, placing prefabs in the project, generating new 3D meshes, creating full scenes, adjusting lighting etc. Since Unity AI assistant is project aware, it can also help developers to fix console errors, debug, modify or write new C# scripts. There are other capabilities of Unity AI such as 2D sprite, texture, animation and sound generation. However, these features are not in the scope of 3D scene manipulation so will not be discussed, but still they are impressive and worth noting here.

Under the hood, the Assistant has three modes that results in different way of act-

ing. These modes are: `/ask`, `/run`, and `/code`. In `/ask` mode, the assistant behaves like a context-aware Q&A system. Using the knowledge of Unity documentation by accessing and retrieving it with Unity's API to answer questions about the project [59]. In `/run` mode, the Assistant converts natural language prompts into desired actions and create C# scripts to achieve them. Once the scripts compile, the Assistant ask for a confirmation and runs them automatically, for example to create gameobjects, change component properties or change the location of a gameobject relative to another. This is the relevant part for controlling 3D scene, all the actions happen with natural language text prompts and user confirmation [60]. In the `/code` mode, the Assistant generates the C# code based on the prompt and puts it into the project folder [61]. It does not automaticaly run the code like in `/run` mode. It complies and test the generated code for syntax error and put it in the project folder. All these modes are managed by cloud hosted language models (GPT-series and Llama-series)[62] that guides Unity Assistant to take correct actions.

Compared to other related systems, Unity Assistant is probably the closest to everyday work of Unity developers in general, not just 3D scene manipulation. LLMR also uses Unity and lets an LLM to make changes in interactive scenes, but it is more of a research framework for mixed reality worlds. It is implemented as a custom pipeline instead of an integrated part of native Unity. Chat-Edit-3D is more generic. It does not require a game engine to run, but interacted via web interface. It works with generic 3D datasets, and using LLMs and visual models to edit 3D scenes. AssistVR [58] is closer in spirit to this thesis because it combines speech input and raycast-based pointing in Unity VR. However, it is mostly an experimental setup to highlight the potential of LLM-assisted interactive systems, rather than a practical assistant to help developers interact with 3D scenes while working within VR.

Relative to this thesis, Unity AI Assistant overlaps the idea of using natural language for scene manipulation, asset creation and help automation. It also shares the goal of reducing the labouring work for developers. Their differences are: Unity AI Assistant uses commercial LLMs, whereas this thesis uses local LLMs accessed through MCP protocol. Unity AI runs in the Unity editor and expects text input, while this thesis keeps the developer inside the VR and uses speech-to-text as the primary input channel to make sure the developer does not leave VR. Finally, Unity AI use API connection for communication instead of a unifying protocol like MCP.

Summary of Unity AI Assistant [57]

- **Main design choice**
  - Integrating the AI as native, commercial-based assistant directly into the Unity editor, using Unity's APIs and three mode interfaces `/ask`, `/run`, `/code`
- **Strong sides**
  - Project-aware editor integration with direct access to scenes, assets, and scripts
  - Can both run scene actions and generate code, all from natural language prompts
  - Broad scope (scene setup, bulk edits, debugging, asset generation) in a familiar editor UI
  - Easy to setup
- **Limitations compared to this thesis**
  - Desktop editor only, no speech input and VR headset specific workflow
  - Limited to commercial models rather than fully local, open source LLMs
  - No protocol layer like MCP, so scene operations are not exposed as customizable tools

Large Language Model for Mixed Reality (LLMR)

LLMR is a framework that lets users create and modify interactive Unity scenes and mixed reality scenes using natural language [10]. User type text prompts that describe worlds and interactions, for example creating rooms with furniture, changing the existing rooms or creating mixed reality scenes. LLMR can both build and edit virtual scenes as well as mixed reality scenes from scratch, which makes it a strong example of language driven 3D scene authoring.

The architecture of LLMR allows it to control Unity through a structured, multiple module code generation pipeline instead of a singe end-to-end model. Each block in the pipeline has well-defined tasks, which makes the system modular. As shown in the LLMR architecture diagram in fig. 17, the system composed of specified GPT modules instaed of one single model. First, the user prompt is fed to the *Planner GPT* which

comes up with the steps to achieve the desired outcome. At the same time, current scene information is extracted as a JSON text in *Scene Analyzer GPT*. The JSON contains the scene hierarchy, with only focussing on the overlapping parts of the user prompt. The plan and the scene information are then combined with a *Skill Library*. Skill library plays a similar role to tool list in MCP. They both expose capabilities to the LLM. Then the plan, scene information alongside with the appropriate skills fed into the *Builder GPT* where the code to achieve user prompt is generated. In the meantime, *Inspector* module iteratively checks the generated code for errors. When the inspector module stops catching errors then the code is complied and executed in the Unity 3D scene producing the desired outcome of ideally the user prompt.

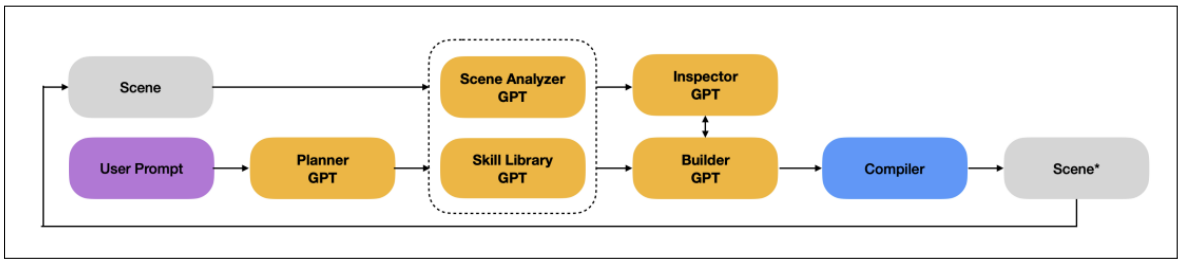


Figure 17: LLMR Architecture (adapted from: [10]).

Compared to the other systems in this section, LLMR, like Unity AI Assistant, runs inside Unity and ultimately changes scenes through C# scripts, while Chat-Edit-3D operates on generic 3D datasets on a web-based chat interface instead of a game engine, and AssistVR focuses on speech input supported with raycast pointing in VR. For this thesis, the key point is that LLMR follows the same overall goal, language based control of Unity scenes, but does so with a completely different architecture that relies on multiple specialized GPT modules, C# code generation, and iterative error checking before the code is executed.

Summary of LLMR [10]

- **Main design choice**
  - Structured pipeline of LLM modules (Planner, Scene Analyzer, Skill Library, Builder, Inspector) that generate, check, and execute C# code for scene changes in Unity
- **Strong sides**
  - Supports rich creation and editing of mixed reality Unity scenes from natural

language
  - Uses planning, scene analysis, and self-checking to reduce errors compared to a single model
  - Received positive feedback for flexibility and usefulness in user studies

- **Limitations compared to this thesis**
  - Text-based, desktop workflow, not an in headset VR developer assistant
  - Controls the 3D scene through generated code scripts, not through a reusable set of exposed tools

Chat-Edit-3D (CE3D)

Chat-Edit-3D is a dialogue-based 3D scene editor where users modify the 3D environments through text chat with an LLM (ChatGPT in this case). The LLM does not edit 3D meshes directly. Instead, it manages a set of vision and geometry models that identify objects, plan edits, and update the underlying scene representation based on the prompt and renders the result [11]. CE3D works with generic 3D datasets instead of a specific engine, hence making it not intractable as the other related works.

Architecturally, CE3D treats a 3D scene as data that can be converted to images (atlases), edit them and merge them back to 3D. The system first renders several views of the scene and sends them to a Hash Atlas network [11]. These atlases are 2D layouts that encode the content of the 3D scene in a way that vision models can process. When user sends a text prompt, the LLM interprets the intent and decides which visual models it needs. Then it selects appropriated models from a *"model zoo"* of 2D and 3D networks, for example segmentation, edge selection or generative models (fig. 18) and calls them with the chosen parameters. This part is similar to usage of MCP tools. Then an executor module run these visual model to modify the atlases, resulting in updated 3D scene. This dialogue of model calling loop can repeat until the LLM decides that the requested edit is finished.

There is a conceptual overlap between CE3D and this thesis in how the LLM works with tools. CE3D's model zoo plays a similar role to MCP tools so that the LLM does not change the scene directly, it decides which external capability to invoke and with what arguments. However, there are different approaches to the same problem. CE3D operates on offline 3D datasets, not on a live Unity VR scene, and all editing

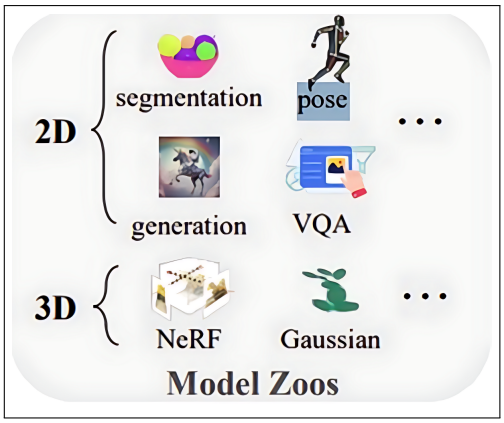


Figure 18: Model zoo in CE3D, showing examples of 2D tools and 3D tools that the LLM can select during scene editing (source: [11]).

happens in the atlas space through image and geometry models instead of engine side operations. In this thesis, Unity exposes concrete scene manipulation actions as MCP tools, the LLM call those tools over a protocol instead of through a custom executor, and edits are applied immediately to a dynamic VR environment instead of a text chat over static scenes.

Summary of CE3D [11]

- **Main design choice**
  - Use an LLM as a chat interface and controller for a model zoo of 2D and 3D vision networks to modify the 3D environments
- **Strong sides**
  - Leverages multi-modal vision models for segmentation, generation, VQA, NeRF and Gaussian splatting, giving rich editing capabilities, also making it easily scalable
  - Combines high level reasoning features of the LLM and low level visual processing of visual models
- **Limitations compared to this thesis**
  - Operates on offline 3D datasets, not on interactive Unity VR scene
  - Interaction is text chat only, no speech or VR workflow
  - Cannot be used for VR development

AssistVR

AssistVR [58] is a virtual scene manipulation study that combines speech input with controller-based raycasting, supported by an LLM (GPT-4o). This study is the closest in input interaction techniques to this thesis, among other related works. However, the main motivation of the authors were to study how people interact with LLM assisted VR editors and what challanges they face. The goal of the paper is less about proposing a new interaction methodology and more about analyzing user strategies when it comes to interacting 3D scenes, difficulties, and design methods to overcome these difficulties for future systems. So they created the AssistVR system with this question and experimental scenario in mind [58]. The experimental scenario is that participants (N=12) are asked to match the original indoor scene to a target scene using both natural language prompts and raycast-based selection. The results highlight the combination of speech and raycast modalities were supported by the majority of the participants.

As for the technical aspect of the study, AssistVR is a multimodal system that accepts speech input and sends it to GPT-4o to extract the intent. They focus on four pre-determined categories "Select", "Deselect", "Modify", and "Undo". The system tries to match the user prompt into these categories. Once matched, they run the pre-defined existing C# scripts in Unity to achieve the desired outcome, such as object selection, material modification, and undo action. If the LLM is not able to match the user intent to these categories, the system sends the entire 3D scene information as context in JSON to guide the LLM to give a natural language explanation or suggestion that is spoken back to the user, but does not directly edit the scene.

The authors ran a user study with 12 participants to analyze interaction patterns, detect common struggles, and design opportunities. They report that multimodal use of speech + raycasting emerged naturally [58]. Participants often used speech for high level commands and raycasting for precise object specification. Users appreciated the efficiency of speech for bulk operations, but also reported issues such as uncertainty about what the model understood, and difficulty phrasing some commands in ways that the model could interpret.

Lack of uncertainty of what the model understood was also a struggle faced in the development of this thesis and the participant's feedback support this limitation. To mitigate this limitation, a confirmation step between the captured speech and sending

the speech to the LLM is integrated in this thesis. Users see the captured speech in a UI panel to make sure that the LLM will receive the intended prompt, and only after confirmation of the prompt it is sent. Furthermore, the bulk operation is also appearing in this thesis, eliminating the workload of unnecessary laboring tasks. Based on these observations, the paper argues for clearer feedback channels, and better transparency about what the LLM can and cannot do.

Compared to this thesis, AssistVR is similar in spirit, since both systems let users manipulate Unity based VR scene through speech combined with controller pointing. However, the technical approach is very different. AssistVR relies on predefined C# scripts to edit the scene, a trained intent classifier and only uses GPT-4o as the language model, and there is no standard tool protocol. In contrast, this thesis uses MCP to expose a wide set of Unity tools as structured actions to a local LLM, and lets the model decide which tool or tools to call for each prompt. Both works support the same overall goal, LLM assisted VR scene editing with speech and raycasting, but AssistVR achieves it through a fixed four category pipeline, while this thesis explores a more extensible, tool based architecture.

Summary of AssistVR [58]

- **Main design choice**
  - Multimodal speech combined with raycast interaction, mapping the prompt into four editing intents that trigger predefined Unity scripts
- **Strong sides**
  - Carefully studied user strategies and challanges, concrete evidence that speech and raycasting complement each other for VR scene editing
- **Limitations compared to this thesis**
  - Editing capabilities are restricted to the four intent categories and set of predefined scripts
  - The system depends on commercial LLM (GPT-4o)
  - The system does not provide a generic, protocol based tool layer that is more scalable and reusable
  - The system was not designed with the idea of a developer helper

Overall, existing work shows that current LLM-driven 3D editors make very different trade-offs. Unity AI Assistant [57] is deeply integrated into the desktop editor and focuses mainly on code and game asset creation workflows, while not supporting VR-specific manipulations right out of the box. LLMR [10] approaches language based control of VR and mixed reality scenes through a multi-stage code-generation pipeline, however it runs on commercial GPT. Chat-Edit-3D [11] gives the LLM control over a model zoo, similar to MCP tools but with visual models, operating on 2D hash-atlas representations of offline datasets rather than dynamic interactive worlds. AssistVR [58] is the closest in interaction style, running in-headset and combining speech with raycast pointing, but its intent-classifier pipeline is narrow and tightly coupled to four predefined edit categories. The system developed in this thesis holds a different point in the design approach and scope. It leverages emerging MCP protocol to expose a reusable set of Unity tools for creation, modification, transformation and other scene operations. Furthermore, it keeps the user in the VR environment with the idea of using it as a developer assistant, uses speech input and raycast-based controllers for editing the scene, like in AssistVR, but relies on local LLMs and the MCP protocol. This protocol-driven, tool layer and local LLM usage focus are not present in the reviewed literature and define the main contribution of the thesis. The main characteristics of the studies are summarized in table 4.

System	LLM deployment	Scene scope	Input modality	Interaction platform	Target demography
Unity-AI Assistant[57]	Commercial models via Unity API services	General Unity projects, 2D/3D scene modification, asset creation	Natural language text prompts in editor	Unity 6.2 desktop editor	Developers, general users, artists
LLMR[10]	Commercial GPT 4 via web API	Scene creation and editing in mixed reality worlds and Unity VR	Natural language text prompts in editor	Unity VR with mixed reality	MR/XR designers, Unity developers
Chat-Edit-3D (CE3D)[11]	Commercial ChatGPT	Generic 3D scenes from datasets, stylized scene creation	Text chat, multi turn dialogue with assistant	Web-based chat interface (2D UI)	3D content creators, graphics/vision researchers
AssistVR[58]	Commercial GPT-4o	Unity based VR scenes for object appearance and component edits	Speech-to-text commands and raycast pointing in VR	Standalone VR headset (Quest 2)	VR users and researchers
Unity MCP VR (this thesis)	Local LLMs via Ollama (Qwen3:8b)	High level Unity VR scenes manipulation with creation, modification and querying of scene objects	Speech-to-text commands and VR controllers for selecting and fine adjustments	Standalone VR headset (Quest 2)	Unity VR developers, users and researchers

Table 4: Summary of related LLM driven 3D scene editing works.

2.6. Novelty of This Work

The background and related work sections showed that LLMs are already used for 3D content creation, scene editing, and VR interaction. However, existing systems usually rely on commercial language models, use custom pipelines that are tied to a specific use case project, or run only on the desktop instead of inside the VR headset. The combination of MCP protocol, local LLMs, and dynamic interactive Unity VR scene manipulation has not been explored in a systematic way. Considering the growing adoption of MCP protocol by major companies, it is timely to study how this protocol can support VR oriented scene editing workflows. Yet, there is still very little work that treats Unity itself as an MCP host-capable application. The following points summarize the novel aspects this thesis introduces to address the mentioned gap.

Local LLMs accessed through MCP

The system runs the LLM (Qwen3:8b) locally on the university server, instead of using commercial model APIs, and connects to it through MCP protocol. This allows data privacy, avoids costly commercial LLMs, and makes it possible to configure the language model and its parameters.

Unity extended with MCP host-like capabilities

In this work, Unity goes beyond its role as a rendering engine and serves as the host-like entry point of the MCP interaction pipeline together with the bridge component. User input starts inside the Unity VR application and translated into concrete scene changes within the same environment. The prompt originates in Unity and ends in Unity in the form of an action. As a result, tool invocation and scene manipulation are tightly integrated into the Unity game loop instead of being handled by an external AI application.

Generic MCP tool layer for scene control

Scene operations such as creating, deleting, and modifying gameobjects are implemented as MCP tool actions instead of predefined C# scripts. This makes the interface of Unity isolated and structured. So each tool can be added, customized, or removed in the MCP server without touching the LLM logic, Unity-side code or the speech layer. As a result, extending, and scaling the system becomes easier, because scene control is isolated behind a clear protocol boundary.

Scene control without leaving VR

High level commands are given by speech, converted to text, and turned into MCP tool calls. VR controllers are then used for selecting and precise adjustment. The workflow is designed so that a developer can stay inside the VR headset while working on the scene, instead of constantly switching back to the PC desktop for adjustments.

Model-independent design

The system treats the LLMs as MCP compatible models that can discover and call tools. Since by nature, MCP protocol is model agnostic, any model that can use tools can be plugged in very easily without redesigning the pipeline, which makes the approach portable across future LLMs and vendors, as well as suitable for various types of hardware requirements. Furthermore, domain specific language models can be chosen for better accuracy. Moreover, alongside with their tool usage capabilities, the language model's plain text response can also be integrated to the developed system. This would allow even more flexibility for domain specific tasks.

In summary, this thesis differs from prior work to create a model-independent LLM assistant by combining local LLMs with MCP to control a dynamic Unity VR scene, exposing scene operations as reusable MCP tools, and integrating speech and controller input for developer friendly editing while the developer remains in the VR headset. These novelties directly address the gap identified previously and form the basis of the research question. The following chapters build on these ideas to motivate the design choices and to evaluate how well this approach works in practice.

3. Motivation and Application Fields

LLMs make it possible to interact with virtual environments, but most of the time, Unity and VR workflows depend on limited interfaces and external commercial services. These constraints restrict privacy, long-term reliability and flexibility. This section talks about the primary motivations behind developing a local, model-independent, and scalable system for near real-time scene control. Then the section talks about the application fields that would benefit from this work. Domains like VR scene prototyping, teaching and training, game development, and research applications would gain new possibilities when scene control is more intuitive, speech-driven, and locally executed.

3.1. Data Privacy

The growing use of AI systems have raised public awareness about how personal data is collected and used. As consumers get more familiar with AI systems, they develop higher expectations for transparency over their data. This tendency is supported by a recent study. A 2025 research [63] on AI acceptance in education found that data privacy concerns play a significant impact in determining whether consumers are willing to adopt AI based systems. This reflects a general shift in user attitudes. People want clear guarantees that their data is handled safely, and they are increasingly cautious about systems that process personal information through external servers.

Commercial LLMs require that user prompts are sent to external providers, and some AI providers could use user data for further model training[64], often without clear disclosure[65]. This creates a clear privacy concern, since prompts may contain personal or sensitive information. In this thesis, the LLM runs entirely locally, so no scene related information is ever transmitted to a third party. The only external dependency is the speech-to-text service provided by Microsoft Azure. This is a practical compromise rather than a perfect privacy solution. Azure receives the raw spoken prompt, and this introduces a privacy dependency that a fully local pipeline should prevent. However, this step does not expose the full interaction history, and Microsoft states that their service is secure and encrypted *"providing multiple layers of security protection across the entire stack"* [66]. This makes this design choice safer in this trade off setting. Future versions of the system can replace Azure with a local

speech model (e.g., OpenAI's Whisper), removing this dependency and achieving full end-to-end privacy.

If data privacy risks are not managed, sensitive VR workflows become exposed. Private conversations with LLMs may leak. Research studies can lose participant confidentiality. Internal company prototypes may unintentionally reveal unreleased products or confidential information. Once this kind of information is exposed, the consequences cannot be undone. Running the LLM locally reduces these risks to a large extent. All reasoning steps, prompts, tool calls, and scene manipulation commands stay on a local machine, and none of this sensitivity data is transmitted to a third party. In this design, only the speech recognition step relies on Azure for practical reasons, while the LLM itself never communicates with an external server. This separation keeps the most critical informations under local control and limits the exposure caused by the commercial services. As a result, the system remains suitable for scenarios that require privacy, and it can move toward full end-to-end private system once a local speech recogniton replaces the remaining commercial dependency.

3.2. Ensuring Long-Term Reliability and Control with Local LLMs

There is much wider options of open source local models than commercial ones, which increases the chance of finding a model that aligns with a specific application. Local models can also be configured, quantized, or fine-tuned to match custom requirements [67], which is rarely possible with commercial commercial LLMs. Small local LLMs can even outperform larger models in domain specific areas when fine-tuned, as this study shows [68]. In some scenarios, including the system developed in this thesis, the task does not require deep reasoning, mathematical precision, or long context handling. The main requirement is stable tool calling and predictable behavior. Under these conditions, small local LLMs are fully capable and perform reliably in near real-time [69], making local inference a practical and efficient choice for near real-time Unity and VR based applications.

Once a local model is downloaded, its behavior becomes fixed. It will not change over time, even if the model family loses official support, it would still work on the downloaded local PC. It can be thought as that the time freezes for the downloaded model.

This makes sure that they will stay usable in the long term. Researchers and developers can count on consistent behavior, reproduce studies, and keep using the same model for years without being affected by external decisions or service discontinuations.

Commercial models, however, can change unexpectedly. Providers can change how models operate, change policies, cut support of older versions, change API schemas, or add new usage guidelines. Any systems that relies on commercial models would be directly affected by these changes. Furthermore, most commercial models are paid services, and pricing can change over time. This creates long term uncertainty for applications that need stability and predictability. Local models do not have these issues, since they are open source (free), operate on the user's machine and are not tied to external providers or changing costs.

In summary, local LLMs provides a level of control and stability that commercial systems cannot guarantee. The wide range of open source models, the ability to fine-tune or customize them for custom needs, and that their behavior stays fixed when they are downloaded give systems a predictable foundation for long-term usage. Also, not depending on commercial services means that unexpected changes in model behavior or prices will not effect the system. These qualities make local models a preferable choice for Unity and VR applications that need to be robust and easy to reproduce over time.

3.3. Model-Independent Design

A model-independent design allows for the system to remain compatible with future LLMs without changing to the communication pipeline. This thesis's plug and play LLM design takes use of MCP protocol's model-independent design, which makes it more flexible. As the LLM ecosystem expands, new models will appear and some will be outdated. By not fixing the system to a single model family or API, any LLM that supports tool usage can be integrated easily. This makes the system sustainable in the long-term and avoids the need for repeated architecture change every time the LLM landscape shifts.

Users have very different hardware capabilities, ranging from low-end laptops to high performance GPUs. A model-independent design allows each user to choose a local LLM that fits their hardware constraints. Lightweight models can be used in low-end machines that do not have a lot of processing power. Larger models, on the

other hand, can be used in high-end machines where more computational resources are available. This flexibility makes sure that the system is available to a wider range of users with different computing needs.

Overall, with model independence, the system can easily adapt to new hardware and LLM technologies, since adding new models does not change the underlying Unity process. Additionally, user can also pick the models that work best with their hardware, so the system can be used with many devices. This flexibility and future-proofing provide long-term maintainability and compatibility.

3.4. Scalability Through Extensible MCP Tools

By adding new tools and actions, the system can be extended. This allows developers to add custom functionality. The newly added tools are discovered and used by the LLM in the inference process. This option to add new tools makes the system horizontally scalable. In fact, scalability is not limited to just adding tools. Entire new MCP servers can be integrated into the system, enabling additional capabilities from different providers or developers. The open source community has already produced several MCP server implementations for Unity [70][71][72][73] and when combined they offer more than 70 tools. In fact, this thesis builds on top of an existing Unity MCP server developed by Barnett and Wu [74]. It used their MCP server communication pipeline and pre-built tools and refined and extended it with custom needs, demonstrating that the system can adopt external MCP components without architectural changes. As more MCP servers appear, the system can expand by incorporating these new servers, improving its functionality and versatility.

Overall, the system's ability to be extanded with additional MCP tools or entire MCP servers demonstrates that its capabilities may expand without increasing architectural complexity. It is possible for developers to integrate community-provided servers or add domain-specific functionality without changing the Unity side logic, LLM inference, or communication pipeline as this thesis demonstrates. By keeping the system architecture and its extensible tool layer separate, functionality can grow with the needs of the project. The MCP ecosystem can naturally grow as new tools and servers appear, preserving the system's adaptability while leaving its fundamental architecture unchanged.

3.5. Reducing Friction in VR Development Workflows

VR development workflows often require constant removing the headset to use PC desktop to adjust objects in the Unity editor and then putting it back on to test the changes. This cycle becomes frustrating over long-term, time-consuming, breaks immersion, and disrupts the developer's focus. Minimizing this constant switching between VR and PC desktop reduces friction and allows the developer to remain inside the scene for longer periods. A speech controlled interaction system improves continuity and minimizes the overall iteration loop.

Most VR content is being created through desktop-based interfaces that rely on complex menus, inspector panels, and mouse and keyboard interactions. Most of the time, these tools were not made for immersive settings, but they remain as the tredditional way for building VR applications. This thesis uses natural spoken language to interact with the scene. It offers a more intuitive option that allows developers to directly express their goals [75]. This way of interacting is typically ignored in existing development workflow, even though it works well for VR [76]. Using natural language input removes unnecessary complexity and makes scene manipulation easier because it is how people would naturally communicate.

The system was designed with developers in mind. When developers use traditional processes, they have to build VR scenes outside of VR and only test them after switching back to the VR headset. This separation makes it hard to test how decisions feel in practice and slows down prototyping. A system that lets developers to create and change the scene directly within VR lowers this barrier [77]. Developers can test the scene as they build it and get visual feedback. This speeds up iterations and supports more intuitive development workflows.

Application Fields

The capabilities of the developed system, make it suitable for a range of practical applications in prototyping, education, game development, and research. Even more application fields will appear as the MCP adaptation increases and more Unity MCP servers come out. The following sections talk about some fields where the system would provides benefits.

Prototyping and Developer Use

VR prototyping often involves switching back and forth between PC desktop and VR headset. Developers requlary leave the immersive environment to adjust objects or test scene layouts, which slow downs the workflow and increase development time. Constant removing and putting on the VR headset also becomes time-consuming and frustrating during long sessions. The system developed in this thesis lowers this friction by allowing developers to create and manipulate scene objects directly within VR using natural spoken language and controller based editing. This allows for quickly try out different scene layouts, gives fast visual feedback, and provides a better prototype session without interruptions.

Education and Training

VR is a popular tool used in educational and training contexts because it allows learners to interact with complex concepts or environments in a controlled and engaging way. A recent study [78] put together industries that benefit from VR trainings and found that VR trainings lead to significant learning results. However, most VR training scenes remain static once the lesson or scenario begins. Instructors or students must exit the headset to modify the environment, change difficulty levels, or introduce new elements. This could slow the learning process and breaks immersion.

The system developed in this thesis removes this limitation by allowing instructors and learners to adjust the scene directly through natural spoken language. A teacher can introduce visual aids, adjust object positions, or change environment conditions during a lesson without interrupting the flow. Furthermore, new training specific MCP tools can be added before the training sessions depending on the needs of the student. This flexibility makes VR based trainings more adaptive and better at meeting to the needs of the students.

VR Game Development

When designing VR games, developers typically try out different ways to setup levels, arrange objects, change the lighting etc. Developers usually use mouse and keyboard for these tasks. But notably, seeing the scene on a desktop editor and in VR are different experiences. This study [79] found that, developers prefer to use the VR

headset instead of traditional workflows when developing VR scenes. But generally, developers have to design the environments on the desktop and switch to VR for testing. This back and forth switching creates an interruptive workflow loop.

The suggested solution facilitates rapid prototyping by letting designers manipulate the virtual environment using voice commands and controllers. Being able to adjust the environment in near real-time speeds up the process of coming up with early level design concepts. Designers can quickly test ideas, get a feel for the scale, and adjust scene layout before committing to the traditional edit workflows. This workflow would support more flexible creativity and makes it easier to transfer design concepts into playable environemnts.

Beyond scene generation, architecture of this thesis also allows the integration of context aware intelligent virtual agents. LLM's plain-text replies can be used for dialogue or behavior generation. This enables context aware intelligent virtual agents that can interpret user intent, reason about scene state, and act meaningfuly in VR environment. This would extend the system's value from development workflows to broader VR based communication and interaction scenarios.

Research and Experimental Studies

The system developed in this thesis could also serve as a practical platform for studying and evaluating tool-using LLMs. Because the architecture is model-independent and directly exposes tool calls through the MCP interface, researchers can integrate different language models without modifying the Unity pipeline. This makes it possible to compare how different language models interpret natural language commands, structure tool calls, and interact with the same set of scene manipulation tools under identical conditions. A similar evaluation is conducted for this thesis and discussed in the Evaluation section 5.2.

Researchers can also use the system to evaluate differences between small and larger models, examine how newer model's tool call structure behave compared to the old models, or test how different model families approach the same task. For example, the system can be used to compare the tool usage patterns of Llama3 and Llama4. There are many studies that try to compare or create a benchmark for evaluating LLMs[80], this thesis could provide a framework for these researches.

Beyond model comparison, the system allows researchers to analyze the structure of

tool calls themselves. This includes studying the JSON message, how models sequence actions when using multiple tools, resolve uncertainty in user instructions, or fail points when encounter with complex prompts. These insights are valuable for understanding the strengths and limitations of tool-using LLMs and for guiding future work in interface design, prompting strategies, and help create better tool schemas.

All the disused application fields show that the designed system is not limited to a single domain but supports a wide range of practical and research oriented tasks. The system can help with VR prototyping, support educational and training scenarios, accelerate level design in game development, and provide a controlled platform for evaluating tool-using LLMs. The system becomes versatile enough to adapt to different requirements across development, learning, and research contexts by allowing near real-time scene manipulation through natural language and keeping a model-independent architecture.

4. Methodology

4.1. System Overview and Architecture

The system developed in this thesis follows a modular, client–server based architecture that connects a Unity VR application with an LLM through the MCP protocol. The used LLM is a Qwen3:8b base model configured with Claude Sonnet 4 system prompt, named **hir0rameel/qwen-claude** [29]. For simplicity reasons, this model is referred to as Qwen3:8b throughout the this thesis. The architecture enables near real-time, natural language driven interactions with various tools with the Unity scene while keeping the language model local and the overall system extensible and model-independent.

At a high level, the system consists of four main components: the Unity application, the bridge component mediating the communication, the MCP server exposing tools for scene manipulation, and the local LLM responsible for interpreting user intent and generating tool calls. MCP server communication structure and base tool schemas is adapted from Coplay GitHub repository [74], and bridge component tool discovery mechanism is taken from Jonigl's GitHub repository [81]. Scene props for testing and evaluation purposes are taken from Unity AssetStore published by AiKodex [82] and Gest [83]. Figure 19 illustrates the complete architecture and the data flow between these core components.

Unity Side

Unity engine serves as the main runtime, interaction environment and the user interface. It serves as the foundation for the rest of the project core components. In this theis `2022.3.51f1` Unity Editor version is used. The engine is responsible for rendering the VR scene, handling VR locomotion and controller input, capturing and storing user speech, and displaying scene changes resulting from tool executions. Unity does not communicate directly with the language model. Instead, the engine side is isolated from the server and LLM logic by using a **bridge** component to handle the communication. For tool executions, Unity listens for incoming requests from the MCP server. It receives the tool command, executes the corresponding action in the scene, and returns the execution result to the server. However, Unity is a standalone program and does not depend on the MCP server to continue operating, it operates whether the tool execution result is successful or failed. Unity is merely a listener for the incoming tool execution requests. This way, Unity side still can perform its operations even if

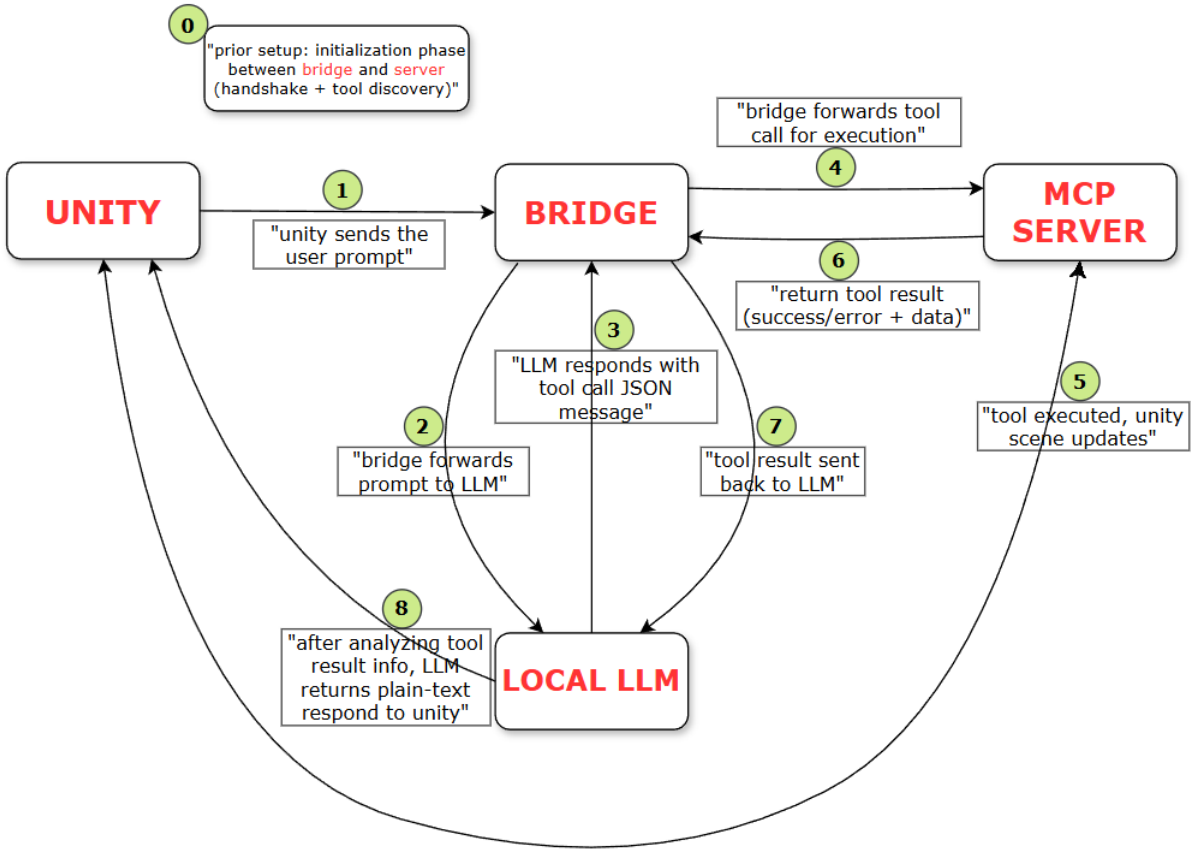


Figure 19: Project architecture.

an error occurs in the server or in the LLM side. This isolation is critical since Unity is the building block of the project and the engine itself offers much more than usage of MCP tools. This separation guarantees that the normal engine functionalities are not affected from the LLM or the server's state.

Unity's interaction with the rest of the system is implemented through a small set of scripts with separated responsibilities. `SpeechRecognition.cs` captures voice input, converts it into text and manages the confirmation workflow before handing the final prompt to `McpPromptSender.cs`. The confirmation workflow step guarantees that the captured speech is indeed what the user said. `McpPromptSender.cs` is the main runtime-side integration point. The script takes the user's confirmed prompt, sends it to the Bridge, and displays visual UI feedback in the VR scene based on the returned response (i.e., success/fail). In the Unity Editor environment, `UnityMcpBridge.cs` listens for and receives structured tool execution commands coming from the MCP server (see fig. 19, step 5). After receiving a command, it forwards it to the proper Unity-side tool handler, typically, `ManageGameObject.cs` script. Finally, this script performs the

requested tool action in the Unity scene (e.g., creating, modifying, or deleting objects), and then `UnityMcpBridge.cs` sends a success or fail result back to the MCP server for further use.

Bridge

The bridge serves as a mediator between Unity, local LLM, and the MCP server. It implements the MCP client logic and handles the communication flow between core components. An important role of the bridge is to prepare the full payload request for the LLM by combining the user prompt with the available tool schemas before sending it to the model. Alongside with the user prompt, it injects all available tool definitions, effectivly consturcting the complete request payload that the LLM recives. This ensrues that the model is aware of which tools exist and how to use them. Once the LLM generates the tool-call, the bridge sends the generated call to the MCP server, and later returns the tool execution confirmation back to the LLM. By coordinating these tasks, bridge decouples Unity from both the LLM implementation and the MCP server backend. As a result, different language models or MCP servers can be replaced without modifying the Unity project itself. The bridge capabilities used in this project are based on the code provided in this GitHub repository [81].

The bridge's interaction with the rest of the system is implemented through a set of scripts with separated responsibilities. `proxy_service.py` is the main orchestration layer, it checks whether the local Ollama model is reachable, if so the bridge forwards the payload (user prompt + tool definitions) to the LLM. Without these tool definitions, the LLM would have no knowledge of available tools or how to use them. It then detects tool-call requests in the LLM response, extracts them from the LLM response message, and help executes those tool calls by forwarding them to the MCP server (see fig. 19, step 4). This way MCP server only recives the tool-call request and not any plain-text that might be generated by the LLM. Once the tools are executed, the bridge then feeds the tool execution results back to the LLM so it can produce a final response for Unity console (see fig. 19, step 8). The bridge practically implements the MCP client logic by connecting to a configured MCP server using `mcp_manager.py` script. It discovers the available tools at startup, and helps invoke the requested tools during the session.

Local LLM

The language model (Qwen3:8b) runs on a local machine and performs two key tasks. First, it interprets natural language prompt provided by the user. Second, it decides whether a tool invocation (single or multiple) is required and generates a structured JSON tool call accordingly. In this pipeline, the LLM is the easiest core component to change. The user can choose any language model, as long as the model supports tool usage, download it, and replace the existing model. While the LLM can run directly on the user computer, it can also run on a remote server (not a commercial cloud server but a remote university server) hosting the LLM. In this thesis, primary deployment configuration is to use remote deployed LLM and access it via VPN connection. As discussed in the previosly in section 3.3, some computers lack the hardware to run some models without overhead, therefore employing a remote server to increase performance is a good optimization. Here the term "local LLM" is used interchangeably with "local" on the user's computer and the university server hosting the LLM locally. Hosting the LLM on the university server can be done by giving its address: `--ollama-url http://10.85.8.40:11434`.

The LLM does not directly execute the actions in the Unity scene itself. Instead, it generates the appropriate tool-call message based on the user prompt, previous message exchanges, and its available tool options and passes it on to the bridge. It relies on the MCP interface exposed by the server to perform the tool execution, ensuring clean separation between reasoning logic and execution.

The LLM is accessed through the bridge as a local Ollama service. The bridge sends the payload prompt to Ollama LLM. The bridge makes the tool discovery before the session begins, and tools discovered from the MCP server. Based on the user prompt, the model then either responds normally, as in a plain-text response without using tools, or returns a structured tool-call request that references one of the available tools and fills the necessary parameters based on the prompt. After the tool is executed and the execution result is returned, the result is appended to the conversation history as additional context, and the model generates a final plain text response regarding the result of the tool execution and delivers the output back to Unity console to inform the user.

MCP Server

The MCP server is a lightweight Python process that runs locally on the user's machine for the duration of a session[74]. It exposes a set of structured tools that MCP clients can call to query and manipulate the active Unity scene. In this thesis, the server is based on the `UnityMCP` implementation from this GitHub repository [84] and is extended with custom functionality (e.g., richer gameobject operations, new actions, conversation history, plug-and-play language model architecture, speech input, raycast-based object selection etc.).

`server.py` is the entry point and tool registry. It contains the tool definitions and expose them for discovery. A separate, dedicated connection layer, `unity_connection.py`, handles communication between Unity and the server. Each tool, like `manage_gameobject`, focuses on a unique type of functionality. The `manage_gameobject` tool is exposed by the `tools/manage_gameobject.py` script which defines the tool schema (parameters, descriptions, few-shot examples, etc.) and translate tool request into Unity commands and send to Unity. On the Unity side, the corresponding tool handler, typically `ManageGameObject.cs`, executes the operation. The execution results are then returned to the server with a success/failure message and any relevant data.

Initialization and Tool Discovery

Before any user interaction takes place, an initialization phase is performed, shown as step 0 in fig. 19. During this phase, the bridge establishes a connection with the MCP server and performs a handshake. The exchange of capabilities, primitives, and other protocol information are exchanged in the handshake phase (see fig. 14). As part of this process, the server exposes its available tools together with their JSON schemas, which defines each tool's name, purpose, and expected parameters to the bridge.

On the bridge side, `mcp_manager.py` script is responsible for connecting to the MCP server and requesting this tool metadata, then storing the discovered tool definitions in an internal list. This list is later provided to the local LLM during inference for it to understand what actions are available and how tool calls must be structured so that the LLM can generate valid tool calls. This initialization step only occurs once at startup and ensures that the system has a shared understanding of tool capabilities before interaction session begins.

Runtime Interaction Flow

After initialization phase and tool discovery are completed, the system operates in normal interaction loop, shown by steps one through eight in fig. 19.

1. **User prompt:** The user issues the prompt through natural speech. After speech-to-text conversion, Unity sends the resulting prompt to the bridge

2. **Prompt forwarded to the LLM:** The bridge receives the prompt, injects the tool definitions and forwards the complete payload to the local LLM

3. **Tool call generation:** The LLM analyzes the prompt based on the intent and previous conversation history and determines whether a tool invocation is required. If so, it responds with a sturcuted JSON message filling the tool parameters

4. **Forwarding tool call:** The bridge forwards the generated tool call to the MCP server for execution

5. **Tool execution and scene update:** The MCP server executes the requested action, which triggers corresponding changes inside the Unity scene. After the execution, Unity sends a success/fail confirmation response back to server

6. **Tool execution result:** After receiving the tool execution response indicating success or fail along with any extra data, the server passes the response to bridge

7. **Result delivered to LLM:** The bridge sends the tool execution result back to LLM. This allows for model to reason about the outcome of the action

8. **Final response to Unity:** Based on the tool execution result, the LLM generates a final informative plain-text response, which is forwarded to the Unity over the bridge and presented to user. Then the user issues a new prompt based on the result, and the pipeline starts again. Note that, although the diagram shows a direct arrow from the LLM to Unity for readability, the response is sent over the bridge.

4.2. Interaction and Execution Pipeline

This subsection explains how the system allows the user to transform their intent into a concrete change in Unity scene. The main interaction components are: VR locomotion

and movement, Controller-based interaction for navigating and object grabbing, VR headset for object selection and Speech capturing. Together these interactions, support uninterrupted work within VR while providing mostly stable and reliable execution, and feedback for each action.

4.2.1. VR Locomotion and Movement

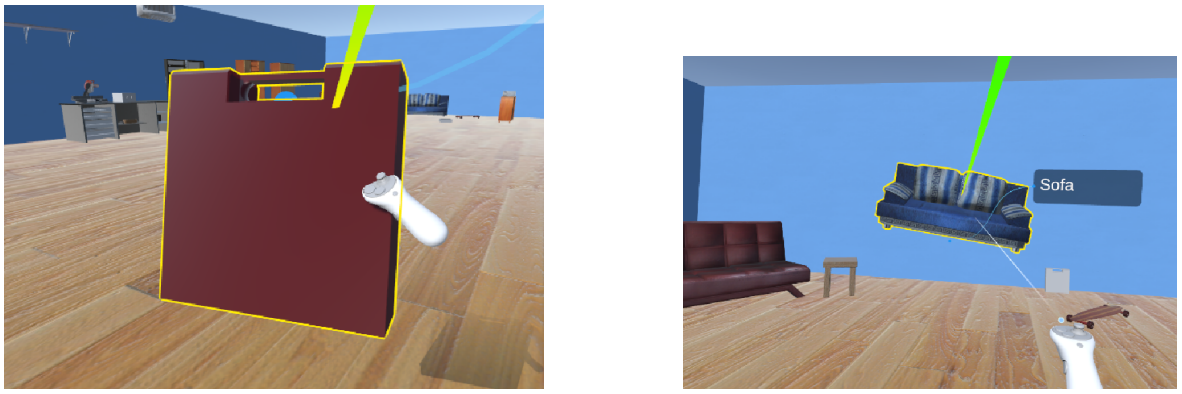
The system begins with the user being able to move, jump, and teleport in the VR environment, these functionalities are adapted from the toolkit [42]. Locomotion allows user to move freely in the virtual scene, which is essential for spatial understanding, object inspection, and getting a sense of scale. In practice, locomotion runs continuously and independently from the bridge, LLM, server operations and speech recognition. While the user moves around the scene using the controllers, the main camera is fixed to the headset position which is essentially the POV of the user. The headset's position becomes the basis for later steps such as raycasting and object detection. Locomotion therefore is not a separate feature, it is part of the interaction context that makes the rest of the pipeline usable.

4.2.2. Controller Input Handling

Controller input provides an intuitive, low-latency interaction method. This thesis uses Meta Quest 2 VR headset and its controllers. The controllers are used for direct object manipulation, and the buttons are used for other features for example, grabbing, starting the speech-to-text process etc. Complete layout of the Quest 2 controller buttons can be seen in fig. 26.

Controller interactions are achieved by using raycast. A raycast is emitted from each controller and intersects with colliders in the scene, resulting in a hit object. This hit object is used for direct operations such as moving, grabbing, or rotating the object. These operations works as follows. Once an object is hit, controller buttons trigger the manipulation. Typical operations include grabbing an object, moving it by moving the controller, and rotating it using the controller's analog stick or bringing the object closer or further. The controllers can also perform the same operations when the objects are at touch distance and in contact with the controllers (see (a) in fig. 20). This way, small tweaks can be made by "touching" the objects that are closer to the camera while raycast is used for further objects. Controller's raycast selection determines which

object is currently "active" for manipulation. This distinction matters because voice commands rely on a different raycast (green colored) coming out from the headset, while grab, move, rotate operations rely on the raycast (gray colored) coming out from the controllers.



(a) Close object manipulation (b) Distant object manipulation

Figure 20: Object manipulation based on distance. Objects within arm's reach (a) are manipulated directly through controller interaction, while distant objects (b) are selected via controller raycast (gray) and manipulated from far.

Another raycast is shot from the headset, near the user's eye level, which determines the currently focused object. Headset raycast is used for providing a concrete reference for language commands. For example, if the user says *"rename this"* or *"delete that"*, the system resolves *"this/that"* using the currently looked object's name. Headset raycast for selecting the looked object also distinguishes hovered objects using an yellow outline shader (adapted from [85]) to avoid accidental operations when multiple objects are nearby. This focus logic is updated continuously, so the pipeline always has an up-to-date reference that follows the user's attention. The system uses both raycast results (controller + headset) to perform the operations. In summary, there are two raycast mechanisms. One coming out from the headset (green colored) which determines the focused object and outlines it yellow. This raycast is used for resolving the objet's names for speech-to-text. Second raycast (gray colored) mechanism is used in both the controllers. This raycast is used to grab, move, rotate objects using the controller.

4.2.3. Voice Input Capture and Speech-to-Text Conversion

The speech pipeline begins when the user initiates voice input through a button press (right controller `"B"` fig. 26). Additionally, for testing purposes, a Unity editor window is implemented, which allows user to send prompts by typing as well as adjusting model parameters. However, this is just for testing and not the actual intended usage, fig. 22 shows this editor window. Audio is captured from the VR headset microphone and sent to speech-to-text conversion. In this thesis, Microsoft Azure* service is used for speech recognition. This introduces a commercial cloud dependency, but it is restricted to transcription only. The resulting converted text is then forwarded to the LLM for further usage and reasoning pipeline.

A key design choice in this pipeline is the explicit confirmation step after recognition and before sending the text to the LLM for execution. Speech recognition can produce errors, and these errors can lead to unintended scene modifications and confuse the LLM context. Instead of immediately dispatching the transcription to the LLM, the system displays the recognized text to the user and requests confirmation (fig. 21). This confirmation is a safety mechanism that reduces accidental tool execution and improves stability during repeated iterative editing.

Figure 21: User confirmation panel. The panel is shown after speech input, requiring explicit approval before execution to prevent errors caused by speech-to-text misinterpretations.

*`https://learn.microsoft.com/en-us/azure/ai-services/speech-service/speech-to-text`

After speech is recognized and before the user confirmation panel is shown, there is a middle step to convert placeholder words *"this"* and *"that"* to the currently looked object. These terms are common in natural language, especially in VR where pointing and looking directly at the objects are part of normal interaction. Without the conversion of these placeholder words, LLM would receive ambiguous instructions and might confuse and choose a wrong target. The system resolves these references using the current looked object derived from the headset raycast. For example:

- *"make this red"* becomes *"make* <ObjectName> *red"*
- *"delete that"* becomes *"delete* <ObjectName>*"*

This substitution is performed locally. The `SpeechRecognition.cs` script simply listens for the words *"this"* and *"that"* and replaces them with the result of the headset's raycast which contains the currently looked object's name. It is deterministic and does not depend on the LLM's interpretation. This design reduces ambiguity and improves reliability because the LLM receives an explicit object identifier instead of a vague pronoun.

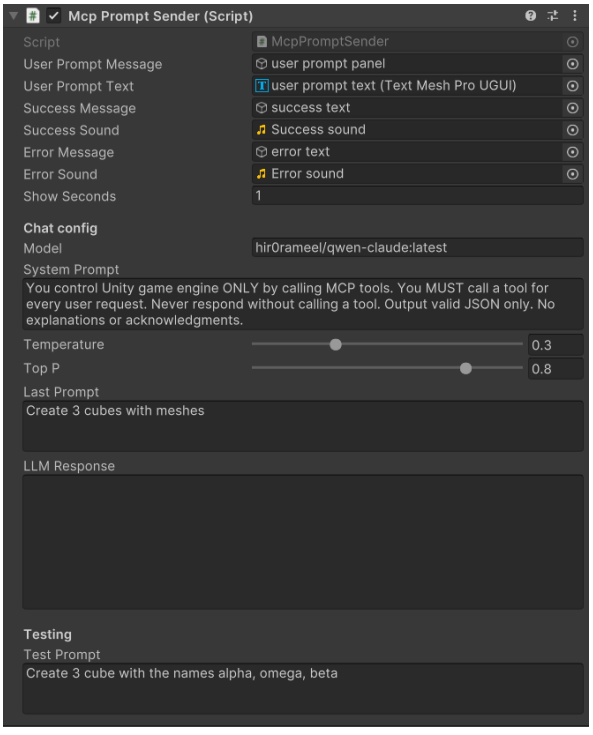


Figure 22: MCP prompt sender window in the editor for manual testing. In addition to speech, the user can type the prompt and change the model parameters in this window.

4.2.4. Context Propagation Across Prompts

The system supports multi-turn interaction by carrying context from previous prompts into the next request. This is required for natural workflows where users refine commands incrementally. This feature allows for quick further follow-up adjustments of the recent actions that took place. Without context, user would need to restate the full instruction every time.

Context propagation is implemented by storing a short history of prior user prompts of 16 messages, corresponding to 8 full dialogues (one user prompt and one LLM response) in a list and sending this list to the LLM every time. When a new prompt is given, it is added to end of the list, and if the list exceeds 16 messages, the oldest message is dropped to respect the context window of the language model and not to confuse the LLM. The exchange number is a variable stored in `maxHistory` and can be adjusted, but through the test, beyond a certain threshold, increasing it does not improve the contextual performance, in fact, when increased too much (>20), performance degrades in this current setting. This context propagation allows the LLM to interpret references such as:

- *"move it closer"* (as in to modify further the last interacted object)
- *"make them bigger"* (as in to reference the last created objects)
- *"do the same for the other one"* (as in to reference the last interacted object)

The context history is kept intentionally small to control performance and avoid sending unnecessary clutter of text. The goal is not to simulate long conversations but to support short, practical editing sequences where consecutive commands are logically connected.

4.2.5. Prompt Construction

Once the recognized speech-to-text is confirmed and contextual references (e.g., "them", "this", "it") are resolved, the system constructs the final prompt that will be sent to the LLM. Prompt construction typically includes the current prompt, conversation history from the previous interactions for context, language model hyperparameter values, and system prompt to guide and encourage tool usage when scene modification is required. All these essentially enrich the user prompt and prepare the ground for the LLM to generate the tool call. This is the current system prompt that the LLM receives:

"You control Unity game engine ONLY by calling MCP tools. You MUST call a tool for every user request. Never respond without calling a tool. Output valid JSON only. No explanations or acknowledgments."

The system prompt is critical because it determines whether the LLM responds with a tool call or with just plain text. In this thesis, the assumption is made that the user's intention is always to ask for tool usage. This is a realistic assumption given the scope of the project. Therefore, the system prompt is structured to encourage tool usage when a prompt is received and to keep responses short and actionable. The final constructed prompt does not embed large scene descriptions. Instead, it relies on explicit object naming, context extraction, and replacing the name of the looked object with raycast for tool calls. After the prompt is constructed, Unity forwards the payload message to the bridge component, which injects the available MCP tool definitions and then forwards the enriched request to the LLM. The LLM analyzes the incoming payload request and tool definitions, which started as just the user prompt but ended up as a much richer message, and generates all the necessary tool calls at once and encapsulates them **into a single message** as an array and sends it back to the bridge. The bridge then iterates over this array and forwards the tool calls to the MCP server for execution.

Eventually, each tool call is executed separately by the MCP server, and their execution results indicating success or failure are returned to the bridge (see fig. 19, step 6). The bridge then forwards these results back to the LLM (fig. 19, step 7), LLM reads the results and produce a final plain-text response summarizing the outcomes to inform the user.

Figure 23 illustrates the prompt construction process from start to finish with a concrete example. When the user speaks the command *"Create a red sphere, name it Kratos"* the LLM translates this natural language request into a structured tool call. The terminal screen in fig. 23 reveals both the request and the result of the MCP communication. The request (highlighted in green) that submitted to the MCP server has the tool name `unityMCP.manage_gameobject` and its arguments: the action type is `"create"`, the color is RGB array `[1, 0, 0]` for red, the object name is `"Kratos"` and the Unity primitive type is `"Sphere"`. After tool is executed, Unity returns a result (highlighted in red) to the server. The execution result includes a success flag, a message verifying that tool executed, and detailed information about the object

that was generated, including its instance ID, transform attributes, and the names of the components that are attached to it. The procedure ends with an `HTTP 200 OK` response, which means that the request-response cycle is over. This status code signals the system that it is ready to accept new user input. No additional user prompts are allowed until this response is received to avoid race conditions.

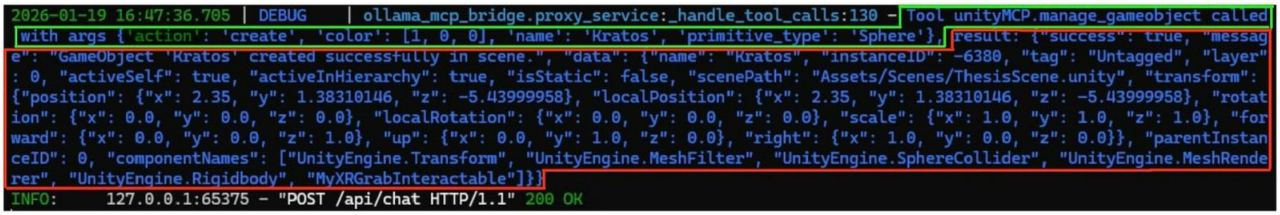


Figure 23: Terminal output showing the MCP tool invocation with request arguments and the corresponding execution result.

4.2.6. Visual Feedback

After the tool execution, the system gives feedback to the user in VR. This feedback is essential to let the user know about the outcome. Correct, partially correct or failed executions could occur and without clear visual confirmations, changes applied elsewhere in the scene could go unnoticed and cause confusion. That is why the user must be able to verify what changed. In this thesis, feedback is provided through multiple channels.

The Unity scene visually updates in near real-time after an object is created, moved, renamed, or deleted. A yellow outline shader is used to indicate the focus target that is being looked at, assisting the user to identfy which item is being referenced. A floating name tag is displayed for the focused object, making object identification explicit and reducing confusion during voice commands. And UI panels show confirmation messages or error messages, for example when a tool call fails or when a command is ambiguous.

This feedback loop is necesarry for effective VR workflows. The user observes the result, adjusts their prompt, and continues with the next command. The usability of the system depends on this feedback loop being fast, reliable and understandable, without the requiring the user to leave the VR environment.

4.3. MCP Server Setup and Tool Schema

This section describes how MCP server is configured and used in the system and what is the tool schema of `manage_gameobject` look like. Instead of covering all MCP features, the section is limited to the topics that are directly relevant to this thesis. These are: server configuration, the primary scene manipulation tool `manage_gameobject` schema, the additional tools catalogue, and the communication flow between Ollama service and the bridge during execution.

4.3.1. MCP Server Configuration

The MCP server is set up through a configuration file. The bridge discovers and connects to MCP servers through a JSON file named `mcp-config.json` (fig. 24). This file specifies which MCP servers are available to the client and where they are located on the machine. As discussed earlier, the configuration file can contain multiple MCP servers with various tool sets. In practice, this allows the bridge component to locate and launch all available MCP servers without hardcoding paths or implementation details into the Unity project. However, in this thesis there is only one MCP server configured, seen in fig. 24. Descriptions of the fields in the config file is given in table 5.

```
"mcpServers": {
  "unityMCP": {
    "command": "C:\\Users\\...\\Python313\\Scripts\\uv.exe",
    "args": [
      "--directory",
      "C:\\Users\\...\\UnityMCP\\UnityMcpServer\\src",
      "run",
      "server.py"
    ]
  }
}
```

Figure 24: MCP configuration file `mcp-config.json`.

At startup, the bridge reads this configuration file, spawns each MCP server process, and establishes an stdio-based communication channel. System becomes more flexible by externalizing server configuration into a dedicated file. Additional MCP servers can be added, removed, or replaced simply by editing this configuration file. This approach avoids tight coupling between the host and specific server implementation and supports scalability without changing to the core pipeline.

Field	Description
`mcpServers`	List of all available MCP servers to connect.
`unityMCP`	Unique name for this specific server.
`command`	Path to the executable program that launches the server.
`args`	Command-line arguments that specify the server's directory and run the entry point script (`server.py`).

Table 5: Description of configuration file fields used in the MCP server setup.

4.3.2. The `manage_gameobject` Tool

The primary used tool for scene control is `manage_gameobject`. This tool's explicit **action** field allows it to enable many scene control actions from a single input point. Instead of using large number of limited scoped tools, `manage_gameobject`'s tool schema defines a set of allowed actions, each with their own custom parameters. `manage_gameobject`'s full set of actions can be found in table 6. All scene operations are combined into a single, standardized tool in this implementation.

This design choice was made for two main reasons. First, it reduces the tool space provided to the LLM. The model only needs to learn how to reliably call a single tool, instead of selecting among many similar tools with overlapping capabilites. Also with more tools, the discovering all of them adds overhead and slows down the configuration of the server. Second, easier extensibility. All scene modifications pass through one handler, which makes it easier to add new actions without dealing with connecting whole new tools to the server. Which ultimately lowers the chance of potential errors.

Action	Description
`create`	Creates a new gameobject in the scene
`delete`	Removes the specified gameobject from the scene
`move`	Changes the position of a selected gameobject, supports relative positioning (e.g., *"above that"*, *"next to this"* etc)
`scale`	Adjusts the scale of a gameobject, accepts natural language commands (e.g., *"make bigger"*, *"double the size"* etc.)
`change_color`	Modifies the material color of a gameobject
`add&modify_component`	Adds a new component to a gameobject or modifies an existing one
`rename`	Changes the name of a gameobject in the scene hierarchy
`duplicate`	Creates a copy of an existing game object, including its components
`light_caster`	Creates a light source in the scene (point, spot, directional)

Table 6: Available scene manipulation actions exposed by `manage_gameobject` tool.

`manage_gameobject` Tool Schema

From the following tool schema, the `manage_gameobject` tool accepts an **action** parameter that determines the operation, alongside with action-specific parameters:

manage_gameobject tool schema

```
manage_gameobject(
    action: str,           # "create", "scale", "move", etc. full list in table 6
    target: str,           # GameObject name or path
    position: List[float], # [x, y, z] coordinates
    rotation: List[float], # [x, y, z] Euler angles
    scale: List[float],    # [x, y, z] scale factors
    color_name: str,       # Named color: "red", "blue", etc.
    scale_command: str,    # Natural language: "make bigger"
    ...
)
```

4.3.3. Other Available Tools

In addition to `manage_gameobject`, the MCP server implemented from this GitHub repository [84], exposes additional tools that provide extra functionality. These tools are included as part of the server's tool catalogue and are automatically discovered during the initialization phase. Complete tool catalogue is given in table 7. While these tools are not central to the VR scene manipulation focus of this thesis, they demonstrate the extensibility of the MCP architecture and provide useful functionality for broader Unity development workflows. This aligns with recent work highlighting MCP protocol's ability to scale horizontally through modular tool and server additions without modifying the existing underlying pipeline [18]. For the purposes of this thesis, these tools are noted but not discussed in detail, because the primary contribution of this thesis is to focus on scene manipulation through natural language in VR. However, as a side note, this modular architecture potentially means future work could leverage these additional tools for more comprehensive Unity Editor control via voice commands.

Tool	**Description**
`manage_scene`	Plays, pauses, saves, and creates Unity scenes, and retrieves the scene hierarchy
`manage_script`	Creates, deletes, and modifies C# scripts in the project
`manage_gameobject`	Scene and object manipulation related actions
`manage_asset`	Imports, creates, modifies prefabs and other Unity assets
`manage_editor`	Controls Unity Editor window and queries project information
`read_console`	Reads or clears Unity console messages
`execute_menu_item`	Executes Unity Editor menu items by path

Table 7: Overview of additional MCP tools available in the MCP server.

4.3.4. Communication Flow and Ollama Terminal Messages

When the LLM deployed on the local PC instead of the university server, the communication between the bridge, the local LLM, and the MCP server can be observed for testing purposes through the Ollama terminal logs (fig. 25). These logs typically show a sequence of `GET` and `POST` requests that represents different stages of the execution process. A healthy interaction consists of one `GET` request followed by two `POST` requests.

A `GET` request means that the user prompt being recieved by the LLM for inference. At this stage, the model analyzes the prompt and decides whether a tool invocation is required. If so, the system issues a first `POST` request that contains the structured MCP tool call. This request triggers an execution on the server, which results in a scene change. After the tool execution, a second `POST` request is sent, indicating that the LLM generated a final plain-text response based on the execution outcome.

This execution sequence can be seen in the terminal logs. Terminal screen can also acts as a useful debugging tool, because it makes it possible to verify each step individually. It can also be used to evaluate performance in terms of execution speed, as each execution step is timestamped. From a system design perspective, this layered communication flow maintains the separation between reasoning and execution that MCP meant to provide.

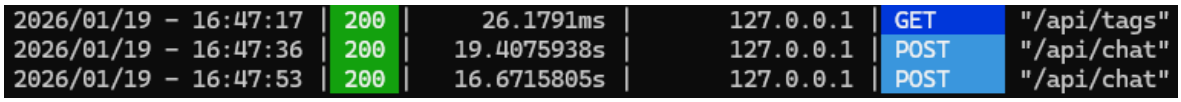


Figure 25: Ollama runtime log showing the sequence of GET and POST requests during a single interaction cycle.

4.4. User Interface and User Experience

This subsection describes how the system presents interaction and feedback, as well as the Quest 2 controllers button layout. The design supports clarity, error visibility, and minimal interruption, so that users can reliably understand what the system is doing at each step of interaction.

4.4.1. Controller Button Layout

The system uses Meta Quest 2 VR headset with its two controllers. Each button is mapped to a specific function to provide intuitive control over the scene manipulation workflow. Figure 26 illustrates the complete button mapping.

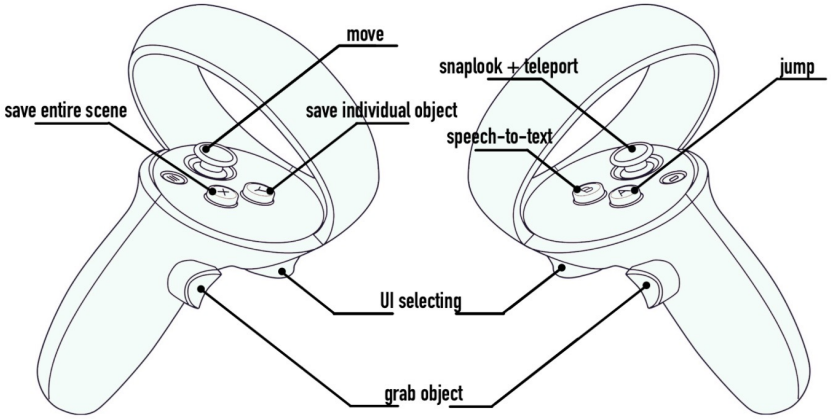


Figure 26: Quest 2 Controller button layout.

This layout intentionally separates scene management functions (left controller) from interaction and input functions (right controller), lowering the learning curve and preventing accidental actions. There is one important distinction to note between the "save individual object" and "save entire scene" buttons. This separation is further discussed in section 4.4.4, but for a quick explanation, the scene state is constantly changing because of for example, physics applied to the objets, position of the camera and controllers, public variable values and so on. So when the user spent some time to change the scene, they need a way of saving these changes so that the changes remain even after exiting the play mode. These buttons give the user this option, to save the entire scene, including the changed camera and controller positions, other objects that might be unintentionally affected from the scene manipulation operations. Or only save the selected objects which for long sessions could be laboring and time consuming, but ensures complete control over the saved scene state.

4.4.2. Visual Feedback Elements

The system uses visual feedbacks to communicate status information to user during the session. All feedback is displayed directly inside the VR environment, making sure that user stays informed without leaving the headset. Figure 27 summarizes the primary

feedback elements used in different stages of the session. As a reminder, another primary feedback element, the user confirmation panel, was discussed in section 4.2.3 and can be seen in this fig. 21.

Error Notification (Figure 27a)

Figure 27a shows the error feedback UI displayed when a request cannot be processed successfully. This can happen, when the model cannot generate a proper tool call, for example by not filling necessary arguments or prompt cannot be resolved to a tool. In this case, the system displays an error warning and suggests to try an alternative phrase. The error notification includes a buzzing sound and haptic feedback for the controllers. This ensures that the user is aware of the failure and feels acknowledged. However, occasionally, the request is correctly handled, but the model still displays an error. This can occur when the server and Unity scene states do not match. As a result, LLM recives a false execution confirmation and show the wrong feedback panel. In these situations, saving the current state and restarting the system helps to decrease these false positives.

Success Notification (Figure 27b)

Figure 27b shows the success message displayed after a tool action correctly executed. This notification is shown when the incoming LLM response contains `"success:true"` keyword (see terminal log result section highlighted in red, fig. 23). A Unity script simply searches for this keyword in the response message and decides to show either success or failed UI panels. The success feedback also includes an audio and a haptic indicators, giving a richer confirmation for the user. However, as previously mentioned in error notifications, sometimes there can be a mismatch between the server and Unity scene. This results in a false success message to be generated. In this situations, the intended outcome is not executed, but the LLM thinks it is and gives a success notification, which triggers the sucess UI in the scene. This Unity scene and MCP server mismatch shows a limitation of the system which is discussed in more detail in section 6.1. It is important to note that in these situations, saving the current scene and restarting the project significantly decrease these false negatives.

Busy State Indicator (Figure 27c)

Figure 27c shows the busy status UI displayed while a previous request is still being processed. During this state, the system stops receiving new input and waits for `200 OK` message (see fig. 23) before continuing to accept new prompts. Attempting to activate speech recognition during this stage, plays a buzzing sound indicating that the input is blocked. This design choice prevents requests from overlapping and race conditions. It also stops the user from submitting multiple commands when they think the system is ready. This reduces uncertainty and communicates that the process is still going on.

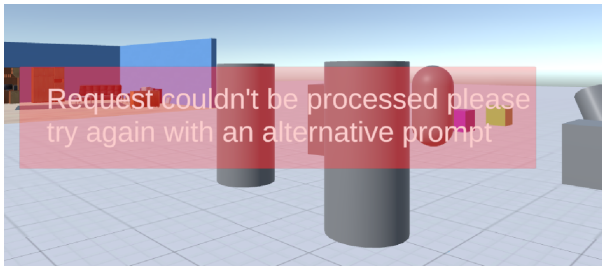
Object Selection Feedback (Figure 27d)

Figure 27d demonstrates the visual feedback used for object focusing. The green ray shoots from the headset and collides with the collider of the object, the yellow outline shader (adapted from [85]) highlights the focused object, and the floating name label (adapted from [42]) displays the object's name (e.g., "Printer"). The floating label names always face the camera, maintaining readability from any viewing angle. These elements indicate which object is currently selected and will be affected by the further actions. This is particularly important when visualizing natural language interactions that include spatial references like *"this"* and *"that"*. When combined, these feedback elements provide a constant visual feedback loop that keeps users informed throughout the session.

4.4.3. Audio and Haptic Feedback

In addition to visual indications, the system uses sound and haptic feedback to validate execution results. Audio signals indicate when speech recording starts and ends, when a tool is executed successfully, or when an error occurs. These indicators provide additional information to the user about the execution results.

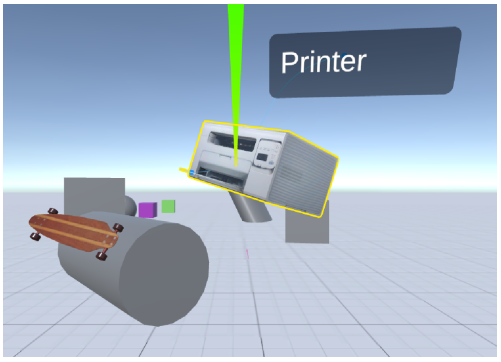
Haptic feedback appears as a short controller vibrations during key interaction moments. For example, when a controller raycast hovers on an object, when an object is saved, or when a tool execution successful or fails. Haptics provide physical confirmation that an interaction has been recorded, which is especially usefull in immersive environments where visual attention may be spread across the scene.



(a) Error message displayed when a request cannot be processed, suggesting to try alternative prompt.

(b) Success message displayed after correct tool execution.

(c) Busy state message indicating that a previous request is still being processed. Preventing to accept new input.



(d) In-VR feedback. Headset raycast (green line), outline shader (yellow outline), and floating name label ("Printer") for the focused gameobject.

Figure 27: UI feedback elements used during an interaction session.

4.4.4. Play Mode Saving

In Unity, play mode introduces a separation between runtime state and editor state. This makes a fundemental problem to save scene changes made during the editting (play mode) session. Without being able to save the changes, upon exiting the play mode all performed tool operations, and scene manipulations will be discarded which makes the scene editting practically trivial. There is no built-in engine functionality to achieve this feature easily. As of January 2026, Unity lists this functionality as *"Under Consideration"* in their officail Unity Roadmap [86].

To implement play mode saving functionality reliably, a third-party package [87] was used and customized to support VR controller input and the choice of saving indiviudal objects or the entire scene during play mode. The package works by capturing the

current state of the scene hierarchy, including gameobject's attached components and serialized properties, and temporarily storing this data outside the scene [87]. Since this storage persist accross Unity's playmode, the package is able to restore the captured objects and their properties back into the scene after exiting the play mode, effectively bypassing Unity's default behavior of discarding runtime changes.

The system supports two saving strategies. Saving the entire scene is available for cases where the user explicitly wants to commit the full current state, for example after completing a big scene edits. However, this approach might be problematic in practice because it also commits to the unintended runtime changes. For example a object falling over because of gravity and not by direct user manipulation. To mitigate this, individual object saving is also supported. In this mode (left controller `Y` buton), only the intentionally selected gameobjects are saved and fixed into the scene.

Developers often experiment freely and only want to preserve successful changes, while discarding trial and error steps. Between these two strategies, there is a trade-off between speed and control. After a long editting session it is faster to save the entire scene with just one button press (left controller `X` button), but it could also save unintended changes. On the other hand, saving each object by hand is labor intensive but guarantees control over the entire scene. This thesis reccomends to save objects individually after each edit and not wait until the whole scene is edited.

5. Results and Evaluation

5.1. Results

This section reports the observed behaviour of the proposed VR scene manipulation system during practical usage. No user study were conducted, therefore the results are captured by testing the system extensively by the author. As mentioned earlier in section 4.1, there are two configurations methods to deploy local LLM. Deploying directly on the user PC, and deploying on the university server. Unless stated otherwise, all results in this chapter correspond to the primary deployment configuration where the local LLM is deployed on the remote university server and is accessed from Unity bridge component over the network via VPN. Secondary configuration, where the same LLM is deployed locally on the user machine is supported by the system but it is not the default basis for the results presented here. The observed results in this section are based on informal, author-driven usage and interaction sessions, rather than controlled benchmarking. Therefore, no statistically significant claims are made. Comparisons under fixed scenarios are presented in Evaluation section 5.2.

Across the testing sessions, the tool call generation mechanism overall remained usable, with occasional reduced reliability during extended interaction sessions. In this work, extended interaction sessions refer to approximately 25 or more consecutive commands in a single session. In such cases, restarting the system restored back the expected behavior of the tool usage reliability. When operating as intended, the system was able to translate user prompts into structured MCP tool calls and apply the corresponding manipulations to the Unity scene with an overall success rate of approximately 91%. Common scene editing operations such as gameobject creation, component modification, and transformation operations were executed reliable enough to use the system for interactive VR interactions. However, perceived delays became more noticeable as the user prompt gets more complex or multiple tool usage were needed for execution. But delays did not prevent the functioning of the system, they just increased the latency.

Speech input was generally reliable as an interaction method. Misrecognition of speech was occurred occasionally, especially under the conditions such as unclear pronunciation, stopping to think in mid-sentence or in noisy environments. To prevent misrecognized inputs to trigger unintended scene changes, the system requires explicit

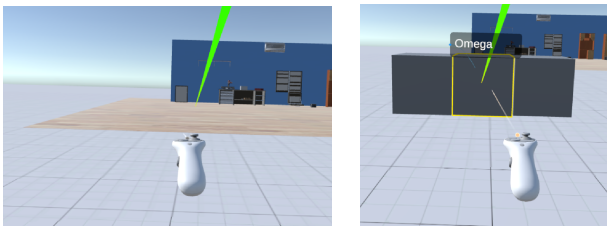
user confirmation of the recognised text before any tool invocation is executed. As a result, misrecognized speeches primarily increase interaction effort through repeated recordings rather than causing unintended modifications.

Finally, several system boundaries were consistently observed. When commands are underspecified or when multiple objects with similar names exist in the scene, the system may apply a valid operation to an unintended target or cannot generate a tool call at all. The system's capabilities are also constrained by the set of exposed tools and their actions, since only the actions represented as MCP tools can be executed. Furthermore, a fundamental system boundary is the LLM itself, since the system is directly bounded by its reasoning capabilities as a language model to generate structured tool calls. Lastly, network connectivity also influences the usability of the LLM deployment, as the execution pipeline depends on a stable connection to the university server and Microsoft Azure services for speech recognition.

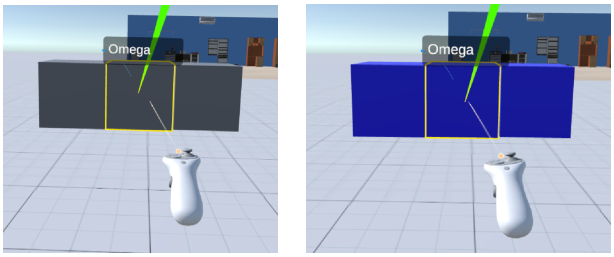
5.1.1. Demonstrated Capabilities

This section provides visual evidence of some of the common actions of the system. The capabilites of the system demonstared thorugh before and after screenshots captured during testing the system in fig. 28. The figures only show typical interactions instead of the full coverage of all actions. These typical interactions are: object creation fig. 28(a), attribute change fig. 28(b), object modification fig. 28(d), and spatial manipulation fig. 28(e). It is important to note that, in spatial manipulation command fig. 28(e), the execution of the request is only partially successful. Although a sphere is created, it is not placed at the intended location between the two referenced objects. This shows the system's limition of handling relational spatial insturctions under certain scene configurations.

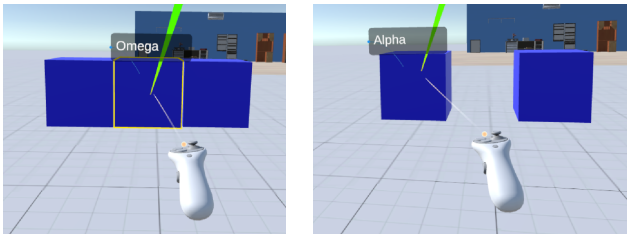
All test results were produced through natural language commands during the runtime. The screenshots confirm that the discussed actions can be performed in practice and the resulting scene state mostly corresponds to the intended user prompts. With the exception of a partial execution(fig. 28(e)), as discussed in section 5.1.2.



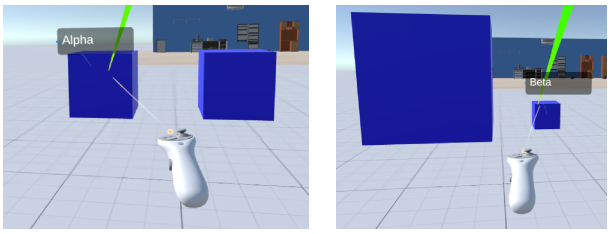
(a) *"Create three cubes, name them Alpha, Omega, Beta "*



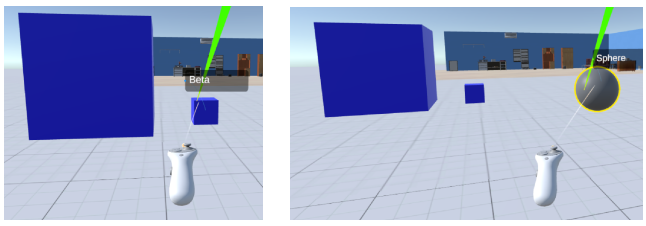
(b) *"Paint them blue"*



(c) *"Delete this (Omega) game object"*



(d) *"Scale up Alpha and scale down Beta"*



(e) *"Create a sphere between them"*

Figure 28: Before and after comparisons of scene manipulation actions performed via natural language. Left images show the scene state before the operation and the right images show the resulting state.

5.1.2. Tool Invocation Reliability

Tool invocations were generally reliable with most user prompts resulted in the correct MCP tool and action being selected and executed. However, different execution outcomes namely, **correct**, **partially correct**, and **failed** executions, were observed depending on the prompt complexity, and duration of the session. One possible cause, in terms of session duration, is that the failed tool calls are more likely to happen in longer sessions and build up over time, which can make it harder to generate new tool calls. By definition, correct executions resulted in right tool and action selection and true user intentions are applied to the scene. Example of correct executions can be seen in fig. 28 from (a) to (d).

During longer sessions, partially correct executions became more frequent, where the correct tool action was invoked but only part of the intended effect was applied. A visual example can be seen in fig. 28(e), where two requests, creation and transformation, were made in a single user prompt, but partially executed. The result is successful creation of the sphere object, but its placement did not fully satisfy the intended spatial constraint (i.e.,*"between them"*). This demonstres that tool and action selection can be correct but the resulting scene modification can remain incomplete, requiring the user to further try to execute the incomplete part.

Failed executions were observed less frequently compared to correct and partially correct executions, but occurred in several different forms. These include cases where no tool invocation occurred at all despite a valid user prompt. And when a wrong tool action was invoked, leading to unintended scene modifications. Unlike partially correct executions, these modifications do not correspond to any aspect of the user's original intent. Another failure mode involved false-positive executions, where the LLM reported successful execution after receiving a success response from the MCP server, but no corresponding scene modification was visible. This problem shows that there can be a mismatch between the execution result from the server and the observed scene state. Finally, sometimes incorrect target selection occurred, most commonly in scenes containing multiple objects with similar names (e.g., "Cube 4", "Cube 5", "Cube 6"). This resulted in correct tool action applied to an unintended object. Failure types were identifiable through visual inspection of the resulting scene state and UI elements.

Overall, the observations show that MCP tool invocations were reliable in many scenarios, but correct execution rates decreases for complex or relational commands,

scenes where objects have similar names, and during extended sessions. In longer interaction sessions, tool calls were more likely to be only partially correct, and failed calls became more likely to appear later in the session. Since earlier execution outcomes stayed in both the scene state and the conversation history, incomplete or failed executions seemed to influence the later commands and reduce overall reliability. Despite this, in practice, restarting the system restored back the expected behaviour and improved tool call generation reliability for the new session.

5.1.3. Near Real-Timeliness and Latency

End-to-end interaction latency, the time it takes to speak the command and see the result in the scene, was evaluated based on four time buckets ranging from 1 to 20 seconds. In this context, "near real-time" refers to the system responding quickly enough to support iterative interaction, rather than strict real-time guarantees. For the tested, realistic usage scenarios, the interaction time from giving a spoken command to observing the resulting change in the scene was typically below approximately 20 seconds, based on repeated and timed tests of the system. For simple commands consist of a single action applied to a clearly identified object, for example, *"duplicate Alpha game object"*, the execution time was typically around 15 seconds. More complex commands, involving multiple actions and relational spatial reasoing, for example, *"Put an orange sphere on top of this cube, name it Foo"* introduced delays, but did not interrupt the overall interaction flow.

While longer response times slightly reduced the sense of immediacy, the system remained usable for iterative scene manipulation. Delays were more apparent during complex instructions, but since the system works on the request in the background, users could continue interacting with other objects in the scene while waiting for the execution of the command.

An important but unmeasured factor influencing overall latency is the dependence on the internet connection when using the default, remote LLM deployment and Microsoft Azure services for speech recognition. When the LLM is deployed on the university server, the system requires an active internet and VPN connection. Similarly, Microsoft Azure speech-to-text services rely on network connection. While network conditions definitely influence overall latency, their exact impact could not be isolated and measured independently and therefore remains unknown.

5.1.4. Limitations and System Boundaries

Speech recognition represents a boundary for the system. Occasional misrecognition of speech, results in repeated input attempts, increasing interaction effort. Speech is the beginning of the execution, so trying to record again slows down the overall interaction.

Unclear or underspecified prompts also form a limitation. When user prompt lack sufficient clarity, the LLM may be unable to resolve the intended action or the target object, failing to generate a tool call. Similarly, scene understanding is constrained when multiple objects with similar names are present. In such cases, when referencing an object, tool operation may be applied to an unintended object. This most likely happens in longer sessions as the default object creation name is the primitive type followed by a number.

Furthermore, the system is fundamentally bounded by the capabilities of the underlying language model. Although Qwen3:8b is powerful and can be used for scene manipulation, the system is ultimately bound by its tool call generation capabilities and reasoing capabilities as a language model.

The system is also bounded by the set of exposed MCP tools and their actions. Only the user prompts that are resolved to the actions represented through available tools can be executed. Which makes the system's ability limited by the coverage of the exposed tools.

Finally, the system depends on internet access to operate. Because both the remote LLM deployment configuration and the speech-to-text service provided by Microsoft Azure require an active internet connection, ideally a fast connection. This introduces an external dependency beyond the system's direct control.

5.2. Evaluation

This section presents a structured evaluation of the proposed VR scene manipulation system using fixed scenarios and controlled conditions to enable fair comparisons across experiments. The evaluation focuses on comparative testing across LLM deployment configurations and different language models. It evaluates scenario-based outcomes, end-to-end interaction latency (measured in time buckets), tool execution reliability (correct, partial, failed), average execution time, and overall task completion. Instead of aiming for statistically accurate benchmarking, the emphasis is on

repeated comparative scenario evaluations to assess practical performance, trade-offs, and alignment with the system's design goals. Throughout this section, two local LLM variants are discussed: the primary model used in this thesis hir0rameel/qwen-claude [29] and shmily_006/Qw3 [88]. These models are based on Qwen3:8b and Qwen3:4b, respectively. Their most notable difference, other than their modelfiles, is that, hir0rameel/qwen-claude uses a Claude-style system prompt, while shmily_006/Qw3 comes with thinking mode disabled by default. For simplicity sake, these models are referred to as Qwen3:8b and Qwen3:4b throughout the evaluation.

5.2.1. Evaluation Methodology

This subsection defines the evaluation methodology and experimental scenarios. To keep comparisons fair, each run uses identical conditions. These conditions are: the same Unity scene, the same MCP server and tool set, and the same 30 natural language prompts in the same order. Conversation history size is varied across scenarios to make two runs in each experiment pair. Complete list of 30 prompts can be found in Appendix A. All tests were conducted by the author, and no statistical significance claims are made. The results reflect system behaviour under controlled scenarios rather than strict benchmarking. The evaluation focuses on four metrics. First metric, end-to-end latency is recorded in four coarse time buckets (<5 sec, 5-10 sec, 10-20 sec, >20 sec). Second metric, tool call reliability, where each prompt is classified as a correct, partially correct, or failed tool call. Third metric, the average execution time of the tool actions. And the fourth metric, overall session success rate which is defined as the ratio of correct plus partially correct executions divided by total prompts. An execution counts as successful if the correct tool action is invoked and the scene change matches the intent fully or remains close but incomplete. An execution counts as failed if it includes missing or wrong tool action invocations, wrong target selection, or false-positive success reports with no visible scene change. Isolated network connection effects, and standalone speech recognition accuracy are not measured and are only reflected indirectly through these outcomes. The following is the detailed summary of the evaluation protocol and scenarios.

Evaluation Protocol

Interaction Scenarios

- **S1:** Conversation history = 16 messages
- **S2:** Conversation history = 8 messages

Measurements

- **M1:** Latency categorized as time buckets

$$<5 \text{ sec} \;/\; 5\text{–}10 \text{ sec} \;/\; 10\text{–}20 \text{ sec} \;/\; >20 \text{ sec}$$

- **M2:** Tool call reliability (correct / partially correct / failed)
- **M3:** Average execution time in seconds
- **M4:** Overall session success rate, expressed as percentage

$$\frac{\text{correct} + \text{partially correct}}{\text{all prompts}} \times 100$$

Experiments

- **E1:** Local vs. remote LLM deployment
- **E2:** Local vs. commercial LLM (text-only input)
- **E3:** Local LLM comparisons among themselves

Each experiment was conducted under both scenarios twice, see fig. 29. All prompts were given via speech input. Only in Experiment 2, instead of speech input, text input is used in order to make a comparison with the commercial LLM since the only way to make a comparison is through text input. Lastly, one message refers to a single turn in the dialogue, either a user prompt or an LLM response. Therefore, a conversation history size of 8 messages corresponds to 4 complete interactions (4 user prompts + 4 LLM responses), and a history size of 16 messages corresponds to 8 complete interactions (8 prompts + 8 responses).

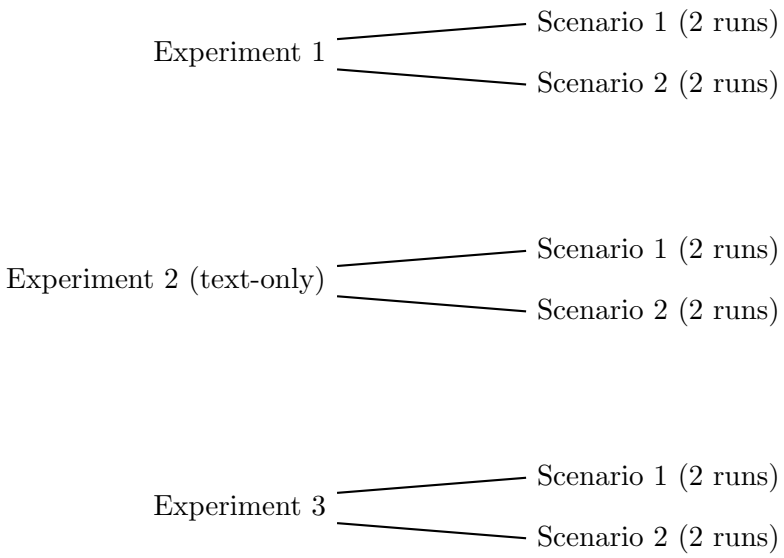


Figure 29: Overview of the evaluation experiments, each executed twice for both scenarios.

5.2.2. Local vs. Remote LLM Deployment

This experiment compares local and remote deployments of the same LLM (Qwen3:8b [29]) under identical conditions. Both configurations use the same MCP server, tool schema, scene setup, and 30 natural language prompts, with only difference being the model's deployment location. In local deployment configuration, the LLM is deployed directly on the author's PC where the Unity scene and other processes also operate. Hardware specs of the author's PC are: Intel i7-10750H @ 2.60GHz, 16GB RAM, 6GB VRAM, NVIDIA RTX 2060. In remote deployment configuration, the LLM is deployed on the university server and accessed via VPN. The hardware specs of the university server are unknown.

Experiment results are shown in table 8 using four metrics. M1 reports how many prompts fall into each time buckets. For example, **1 / 4 / 24 / 1** reads as 1 prompt executed under 5 seconds, 4 prompts executed under 10 seconds, 24 prompts executed between 10-20 seconds, and 1 prompt executed in more than 20 seconds. M2 captures tool call reliability as correct, partially correct, or failed executions and shows how many prompts fall into each category. For example, **25 / 2 / 3** reads as 25 correct executions, 2 partially correct executions, and 3 failed executions. M3 reports average execution time per prompt. M4 reports overall session success rate.

Tool call reliability and session success rates are similar between local and remote

deployments across both conversation history lengths. In all tests, success rates remain between 90–93%, indicating that deployment location does not affect the correctness of tool usage. However, the latency measures (M1 and M3) show a significant difference. The local deployment has significantly longer average execution times, close to 2 minutes per execution. On the other hand, remote deployment generally has an average execution time of about 15 seconds. This latency difference is primarily caused by local PC limits and system overhead.

From a usability perspective, remote deployment is recommended for this thesis because it provides significantly better responsiveness while being reliable in tool calls. Additionally, remote servers also allow the use of larger and more capable language models that may be infeasible to run on a PC. While this is the case, accessing to such servers is often limited to university or enterprise environments, making it less practical for everyday users and developers. Overall, local LLM deployment remains as viable option, but remote LLM deployment offers a clearly superior user experience for near real-time VR interaction.

Scenario	**Deployment**	**M1**	**M2**	**M3**	**M4**
Scenario 1 (conv. history = 16)	Local (Qwen3:8b[29])	0 / 0 / 0 / 30	27 / 0 / 3	113 s	90%
	Remote (Qwen3:8b[29])	0 / 7 / 23 / 0	26 / 2 / 2	14.8 s	93%
Scenario 2 (conv. history = 8)	Local (Qwen3:8b[29])	0 / 0 / 0 / 30	26 / 2 / 2	116 s	93%
	Remote (Qwen3:8b[29])	1 / 4 / 24 / 1	25 / 2 / 3	15.2 s	90%

Table 8: Evaluation results of local vs. remote LLM deployment configurations. M1: latency as time buckets (<5s / 5-10s / 10-20s / >20s). M2: tool call reliability (correct / partial / failed). M3: average execution time per prompt. M4: overall session success rate.

5.2.3. Comparative Evaluation of Local vs. Commercial LLM

This experiment compares Qwen3:8b[29] and Claude Sonnet 4, under identical conditions. Sonnet 4 is accessed via Cursor IDE which acts as the MCP client. Sonnet 4 is chosen as the base commercial model because it outperformed other commercial models (GPT-4.1, Sonnet 3.5, GPT-4o) in various testings. The evaluation is based on the same 30 natural language prompts executed in the same order, scene configuration, MCP server, and tool schemas.

The comparison is conducted in text-only mode. Sonnet 4 accessed via Cursor does not support speech input and therefore cannot be used inside the VR headset. As a result, commercial models are not suitable for the intended VR workflow of this thesis. But they are included in the evaluation section as a strong reference baseline for tool use. To use text input instead of speech input in the thesis system, speech-to-text pipeline is bypassed and text input used directly thorugh Unity editor window fig. 22.

Results are summarized in table 9. Across both scenarios, Sonnet 4 achieved consistently higher tool call reliability, lower latency, and perfect session success rates. Qwen3:8b remained usable, but showed occasional partial and failed executions, leading to slightly lower success rates. Differences between the two conversation history sizes for Qwen3:8b, were observed limited.

Scenario	**Type**	**M1**	**M2**	**M3**	**M4**
Scenario 1 (conv. history = 16)	Local (Qwen3:8b[29])	1 / 2 / 27 / 0	26 / 2 / 2	15.1 s	93%
	Commercial (Sonnet 4)	0 / 21 / 9 / 0	30 / 0 / 0	8.9 s	100%
Scenario 2 (conv. history = 8)	Local (Qwen3:8b[29])	0 / 4 / 26 / 0	27 / 2 / 1	14.5 s	96%
	Commercial (Sonnet 4)	2 / 19 / 3 / 0	30 / 0 / 0	8.7 s	100%

Table 9: Evaluation results of local vs. commercial LLM. M1: latency as time buckets (<5s / 5-10s / 10-20s / >20s). M2: tool call reliability (correct / partial / failed). M3: average execution time per prompt. M4: session success rate.

5.2.4. Comparative Evaluation of Different Local LLMs

This experiment compares Qwen3:8b (hir0rameel/qwen-claude:latest[29]), and Qwen3:4b (shmily_006/Qw3:latest[88]) LLMs under identical conditions, where the only varying factor is the language model itself. Before deciding these models, more than 35 LLMs from the Ollama and Hugging Face libraries were tested. All the testel LLMs are in Appendix B. However, over 80% of these models either did not support MCP tool usage or produced unreliable tool calls, making them unsuitable for near real-time scene manipulation applications. Among all tested models, Qwen3-based models consistently showed the strongest performance. Interestingly, model size did not correlate to better tool usage behavior. In several cases, bigger Qwen3 variants, for example Qwen3:32b, performed worse than smaller models in identical MCP server, scene and tool schema conditions. This indicates that tool usage is influenced more by model configuration and system prompting than its size. Based on the observations, the evaluation focused on the two strongest candidates, Qwen3:8b and Qwen3:4b. Both models were tested using the same MCP server, identical tool schemas, the same scene setup, and the same 30 natural language prompts executed in the same order. The results are summarized in table 10.

Across both scenarios, Qwen3:8b clearly outperformed Qwen3:4b, achieving higher tool call reliability and higher session success rates making Qwen3:8b the primary LLM of this thesis. It is also worth noting that Qwen3:4b was unstable in Scenario 1 for unclear reasons. In Scenario 1, it required multiple runs to obtain usable results. Additionally, in Scenario 2, Qwen3:8b produced some false-negative executions. In these cases, scene modification was applied correctly, but the UI reported an error. This shows that there sometimes is a mismatch between the MCP server response and resulting scene changes. However, interestingly, false-negative executions were generally faster than true-positive executions. The most probable explanation is that, because the LLM writes shorter response messages in false negative cases, it writes them faster. And also Unity detects correct or failed execution by searching for the `"success:true"` keyword in the response message (see terminal log, fig. 23), it spends less time searching for the keyword in a shorter response message.

Overall, Qwen3:8b [29] achieved the most reliable performance among the tested local LLMs. It showed higher tool call reliability and success rates while maintaining acceptable near real-time latency, making it well suited for MCP-based tool usage and

VR scene manipulation. Hence it is used as the primary language model of this thesis.

Scenario	LLM	M1	M2	M3	M4
Scenario 1 (conv. history = 16)	Qwen3:8b [29]	0 / 4 / 26 / 0	27 / 2 / 1	13.3 s	96%
	Qwen3:4b [88]	1 / 10 / 19 / 0	15 / 3 / 12	10.1 s	60%
Scenario 2 (conv. history = 8)	Qwen3:8b [29]	3 / 19 / 8 / 0	21 / 2 / 7	8.8 s	76%
	Qwen3:4b [88]	4 / 15 / 11 / 0	16 / 2 / 12	8.84 s	60%

Table 10: Evaluation results of local vs. local LLMs. M1: latency as time buckets (<5s / 5-10s / 10-20s / >20s). M2: tool call reliability (correct / partial / failed). M3: average execution time per prompt. M4: overall session success rate.

5.3. System's Alignment with Design Goals

This section evaluates how well the proposed system achieved the design goals outlined in the Novelty section 2.6 and the Motivation section 3. Each goal is classified as **met**, **partially met**, and **not met**, based on the observed system behaviors in the results and evaluation sections.

Data privacy

Not met. While the local LLM contributes to data privacy, full data privacy is not achieved because of the dependency on the Microsoft Azure speech recognition service. Which requires transmitting user speech to an external service.

Near real-timeliness

Partially met. End-to-end interaction latency averages around 15 seconds, which is usable for interactive workflows, but leaves room for improvement and tends to be more obvious with regular use.

Model-independent design

Met. The system has a plug-and-play architecture that allows different tool-using LLMs to be integrated without modifying the core pipeline, enabling a wide range of

models to be used.

Natural language driven VR scene manipulation

Met. The system successfully translates natural language prompts into MCP tool calls and supports a wide range of scene manipulation actions with mostly reliable execution.

Reduced friction in VR development workflows

Met. By allowing developers to remain inside the VR headset while modifying scenes, the system reduces constant switching between VR and desktop-based tools.

Scalability through extensible MCP tools

Met. The MCP-based design supports the addition of new custom tools and even separate MCP servers, allowing the system to scale easily in functionality without architectural changes.

6. Discussion

6.1. Interpretation of Key Findings

The results indicate that integrating LLMs with MCP protocol in Unity enables to manipulate and control VR scenes directly through natural speech without leaving the VR headset, with a acceptable level of reliability. This finding is closly related to the research question specified in the Introduction section 1. As a remider, the research question was:

> ***RQ:*** *How can LLMs be integrated with MCP protocol and Unity to enable developers to manipulate and control VR scenes directly through natural speech, without leaving the VR environment?*

The results imply that tool-using LLMs **can** function as practical scene manipulators in virtual environments. By translating user prompts into structured tool calls quickly enough to be used in near real-time applications. They behave as probabilistic assistants whose outputs must be interpreted, verified, and occasionally corrected. This occasional need for correction is most notable in false-positive and false-negative cases. False-positives occur when no tool execution happens, but the LLM thinks the tool executed correctly. False-negatives occur when a correct tool is executed but the LLM thinks that no tool is executed. These situations indicate there can be a mismatch between the MCP server and the Unity scene. Their impact affects user behavior in two different ways. Since false-negatives still result in tool execution, the user can ignore the false error UI panel and continue with the interaction session without much effect, they are often tolerable. But false-positive errors are much more critical. The user might think that the tool execution has indeed occurred but only partially, making them difficult to differentiate from actual errors. This might result in repeated execution attempts for the same prompts, reducing the user experience. Or unintended manipulations to take place when reattempting the prompt.

Furthermore, the evaluation results indicate that the most influential component for the performance of the system is by far the LLM itself, then deployment configuration, and lastly conversation history length. When the same LLM deployed in the remote university server versus in the local PC (see table 8), even though the local PC executions in general took around 2 minutes to complete compared to 15 seconds in the

remote server, their success rate was practically identical. This indicates that, in terms of tool usage, the LLM capabilities are ultimately the determining factor, which is not surprising.

In terms of usability of the system, the most influential metric was the latency. Latency is mostly affected by the deployment configuration of the LLM, and even though the LLM has a similar success rate across deployment configurations, as the latency increases, the system becomes practically trivial to use because of long wait times inside the virtual environment. Conversation history length did not seem to affect the success rate of tool executions. This is probably because, realistically, most of the context requiring follow-up refinement commands is at most two or three prompts back, and for further in the history, raycast-based name replacement is preferred.

A notable and unexpected results when testing the LLMs, was that larger models and thinking-enabled models did not produce better tool calls than smaller models, which is counterintuitive. In several cases, they performed poorly to a point that they were not feasible to use for scene manipulation. Even `gpt-oss:120b`, the largest model tested, did not produce reliable tool calls in multi-step interactions. The trend with the bigger models was that they generally stopped generating next tool calls after the first tool call is executed, when requiring to use multiple tools.

The thinking mode enabled LLMs also performed poorly because of the extra tokens used in the thinking process. This resulted in an increase in latency and filling up the context window quicker, especially in smaller models, making them practically unusable in this thesis setting. This suggests that, for tool-based scene manipulation, alignment with structured tool usage and execution time is more important than model size or thinking ability.

Overall, the evaluations and results findings support the research question by showing that LLMs can be integrated with MCP and Unity to enable practical, near real-time, speech-based control of VR scenes without leaving the VR environment.

6.2. System Design Reflection

The observed evaluation results are direct consequences of the intended system design, which priotrizes reliable tool executions. The user can manipulate the correct intended scene object because of how the user prompts are grounded before being passed to the LLM. Vague references are resolved to their object names, and these object names

are passed to the LLM, since tools communicate through object names in the scene hierarchy. Relying on object names creates a trade-off for simplicity, because it introduced the risk of name collision (e.g., Capsule 2, Capsule 3 etc.). Instead, a potential improvement could be to use unique object IDs, which would eliminate the accidental name overlaps. While this idea was explored, it is ultimately discarded because a later-implemented naming convention already provided mostly reliable outcomes without increasing system complexity.

The system is designed to wait for the execution to happen before accepting new user input. This design choice may limit the rate of commands issued but is necessary for not to confuse the LLM and mixup prompts. The choice also increases the chance of proper synchronization with the MCP server about the result on tool execution. A potential improvement could be to implement a queue that stores the next commands while the current one is being processed.

Tool templates are optimized with natural language interaction in mind, therefore they expect common human-like interaction patterns. As a consequence, some of the supported vocabulary remains discrete, which can introduce uncertainty. For example, *"make it bigger"* does not necessarily reflect the actual intent of the user. The magnitude of *"big"* is unclear. Potentially a confirmation mechanism can be implemented to double-check for ambiguous adjectives and ask for more specific input. However, the design choice favors maintaining conversational flow over constant strict parameter confirmations.

Set of predefined tool actions were chosen and clearly separated for a range of predictable scene manipulation actions. Compared to another approach from LLMR [10], as discussed in related work, where their system generates and executes free-form C# code for scene control. Using tool-based interaction allows for more deterministic choices for the LLM, whereas for the code generation approach, it is much more prone to errors.

During the practical usage of the system, partially correct executions were treated as successful executions when the intended action or object was correct but the placement was incorrect. In practice, such cases did not significantly affect technical usability, because VR controllers allow for fast and precise transform manipulations. The low correction effort for misplacement therefore makes the partially correct executions to count as successful. However, in some cases misplaced objects were created outside the

user's direct field of view, which could temporarily confuse the user. This highlights a need for better visual indications for partially correct executions.

Overall, prioritizing reliable tool execution and low-effort correction resulted in a system that is usable in practice, even in the presence of partial executions.

6.3. Limitations

Several limitations affect the current system and its general applicability, and should be considered when interpreting both the evaluation results and the overall system design. First, feedback clarity remains a limitation for partially correct executions, especially when the resulting object may be placed outside the user's immediate field of view. In such cases, users may be uncertain whether a tool action was executed correctly, partially, or not at all. Previously discussed related work [58], on speech-based VR scene control, similarly highlights the importance of clear feedback. In the research, they hold a user study and find that users interacting with their system (which is similarly speech-controlled LLM-assisted 3D scene editing system) were often confused when commands did not produce the expected outcome and when the system failed to provide clear feedback explaining the cause of failure. Additionally, study participants suggested that visual feedback could be further improved by introducing explicit indicators, such as adding a progress bar, which can directly be taken as a recommendation for this thesis. About the partially correct executions, false-positive and false-negative outcomes form a broader limitation category, because occasional mismatches occur between the MCP server's reported status and the actual scene state. The lack of confirmation mechanism makes it difficult to ensure that an intended modification has indeed been made correctly.

Another limitation is about scene consistency during play mode saving. Saving the entire scene at runtime can unintentionally capture elements that are not part of the intended scene. For example, the saved state may include the XR rig's latest position, effectively changing the starting point of the system on reload, or temporary helper objects such as teleportation UI elements, raycast visualizers, or other interaction aids that exist only during gameplay. A more selective and intent aware runtime saving system was not within the scope of this thesis and represents a complex problem that would require dedicated system design and evaluation beyond the current work. The saving system used in this thesis is adapted from this [87] third party package.

The evaluation expereimets are also limited in scope. The system was not evaluated with a large-scale user study and was only tested in a limited number of scenarios. While the reported results show technical usability, they do not capture how real developers or researchers with different workflows would interact with the system over extended periods. As a result, claims about developer productivity, long-term usability, learning curve, and adoption in professional development workflows remain limited without direct feedback from a broader user population.

The system is further limited by the capabilities of the language model and the available hardware. Even though Qwen3:8b is relatively a small model, tool execution latency is directly related to remote server hardware where the model is deployed. Furthermore, debugging also remains difficult because of the distributed nature of the pipeline, which includes LLM inference, bridge component, tool routing, MCP server-side, and Unity scene updates. This makes it difficult to pinpoint any root cause of failures. Finally, the reliance on Microsoft Azure for speech recognition introduces an external dependency that affects fully offline usability and weakens end-to-end privacy guarantees.

6.4. Future Work

Several directions for future work and research oppurtunites emerge naturally from the mentioned limitations. Feedback clarity for partially correct executed commands could be improved by introducing visual hints in VR. For example thorugh a directional arrow UI or camera movements that hint where and what changed.Also, adding a security mechanism that detects inconsistencies between server response and actual Unity scene updates might make the system more reliable.

New studies may also focus on more detailed evaluations and measurments. Adding new metrics that isolate possible errors at different points in the pipeline, would help figure out where executions fail. Additionally, conducting a large scale user survey involving developers and researchers could provide useful insight about how well the system works in real world situations.

The plug-and-play LLM architecture of the system allows future extensibility. As tool-using language models continue to improve, newer and more capable models can be added without redesigning the pipeline. This could potentially reduce the current model and hardware related limitations. 0ne of the most important future work would

be to implement a fully local speech recognition pipeline. This would eliminate the only commercial dependent part of the system and achieve full data privacy. Replacing the current Microsoft Azure service with a local speech-to-text, such as OpenAI's base Whisper model [89], could be a good opiton.

Finally, future research could explore new tool invoking strategies. LLM's streaming capability could enable **streamed tool-calls**, allowing tool parameters to be sent incrementally as they are generated, instead of all at once. This approach is already being investigated in recent work on streaming function calls [90] and could be interesting to apply the tool invoking pipeline of this thesis. Moreover, while thinking-enabled models did not improve tool-call quality in this thesis, future work should look into separating thinking part from tool-call message generation. Recent state-of-the-art research [91] shows that this decoupling can improve both response time and reliability.

7. Conclusion

This thesis explored how tool-using LLMs can be integrated into Unity through the MCP protocol to enable in-VR scene control using spoken language. To achieve this, an interactive system architecture was developed that translates spoken input into scene manipulation actions directly inside the VR environment.

This architecture connects a Unity-based VR application to a local, tool-using LLM through a bridge component that communicates with the MCP server via the MCP protocol. The LLM interpreted the natural language prompts and converted them into structured tool calls that modified the VR scene in near real-time (approximately 15 seconds), achieving an average tool-use success rate of approximately 91%. Scene manipulations were handled using the MCP protocol's structured tool-call mechanism, which resulted in deterministic and mostly reliable interactions with the scene. This tool-based approach enabled a modular and extensible architecture by allowing new custom tools to be added without modifying the core system, while remaining compatible with various tool-using language models.

Overall system usage and controlled evaluation scenarios demonstrated that speech-driven, tool-based scene manipulation in VR was feasible for interactive scene manipulation tasks. The results showed that the system's usability was primarily influenced by interaction latency, feedback clarity, and the effort to recover from partially correct executions and errors. A key finding was that larger and/or thinking-enabled language models (>20b) did not produce more reliable tool calls and often introduced additional latency, making them less suitable for interactive VR workflows. In contrast, smaller models (<8b) were more suitable in practice for VR interactions, but this reason should not be attributed to model size alone. Instead, the results suggested that internal system prompt design and parameter configurations play a more important role in tool-use performance. This case is further supported by observations within the same model family, where a smaller, modified variant Qwen3:8b[29], consistently outperformed its larger, unmodified counterpart Qwen3:32b. This suggests that system prompt tuning and parameter configurations may have a stronger effect on tool-use behavior than model size alone.

The developed system showed that natural speech can be used as a viable input method for near real-time scene control tasks in VR. This reduces the need to switch

between the desktop and the VR headset and support more intuitive, immersive workflows. Beyond technical feasibility, the system contributes to ongoing research on tool-using LLMs by making tool executions visually observable, hence making the architecture a practical framework for analyzing and comparing tool-use behavior across models. This demonstrates how the system's modular architecture can increase transparency, scalability, and model comparison in interactive environments.

In summary, this thesis demonstrated that combining tool-using LLMs with MCP servers and VR creates a new kind of immersive authoring system where the developers can create and modify objects directly inside the environment they are building. Despite its limitations, the system serves as a promising foundation for LLM-integrated VR development workflows. It also supports future research on speech-driven interaction, embodied AI, and tool-using LLMs.

References

[1] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.

[2] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.

[3] S. G. Mohan. The 3 horizons of llm evolution. *KDnuggets*, May 2025.

[4] Aaron Parisi, Yao Zhao, and Noah Fiedel. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*, 2022.

[5] Johann Wentzel, Matthew Lakier, Jeremy Hartmann, Falah Shazib, Géry Casiez, and Daniel Vogel. A comparison of virtual reality menu archetypes: raycasting, direct input, and marking menus. *IEEE Transactions on Visualization and Computer Graphics*, pages 1–15, 2024.

[6] Model Context Protocol. Architecture—model context protocol specification. `https://modelcontextprotocol.io/specification/2025-11-25/architecture`, 2025.

[7] Model Context Protocol. Architecture — model context protocol specification. `https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle`, 2025.

[8] A2A Protocol. Agent skills & agent card — python tutorial. `https://a2a-protocol.org/latest/tutorials/python/3-agent-skills-and-card/#agent-card`, 2025.

[9] Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao Cai, Binbin Chen, Xuan Xie, and Yin Zhang. Agent network protocol technical white paper. *arXiv preprint arXiv:2508.00007*, 2025.

[10] Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. Llmr: Real-time prompting of interactive worlds using large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–22, 2024.

[11] Shuangkang Fang, Yufeng Wang, Yi-Hsuan Tsai, Yi Yang, Wenrui Ding, Shuchang Zhou, and Ming-Hsuan Yang. Chat-edit-3d: Interactive 3d scene editing via text prompts. In *European Conference on Computer Vision*, pages 199–216. Springer, 2024.

[12] Tong Xiao and Jingbo Zhu. Foundations of large language models. *arXiv preprint arXiv:2501.09223*, 2025.

[13] Andreas Jungherr and Damien B Schlarb. The extended reach of game engine companies: How companies like epic games and unity technologies provide platforms for extended reality applications and the metaverse. *Social Media+ Society*, 8(2):20563051221107641, 2022.

[14] Ayah Hamad and Bochen Jia. How virtual reality technology has changed our lives: an overview of the current and potential applications and limitations. *International journal of environmental research and public health*, 19(18):11278, 2022.

[15] Lin Geng Foo, Hossein Rahmani, and Jun Liu. Ai-generated content (aigc) for various data modalities: A survey. *ACM Computing Surveys*, 57(9):1–66, 2025.

[16] He Yan, Xinyao Hu, Xiangpeng Wan, Chengyu Huang, Kai Zou, and Shiqi Xu. Inherent limitations of llms regarding spatial information. *arXiv preprint arXiv:2312.03042*, 2023.

[17] Model Context Protocol. Model context protocol documentation. `https://modelcontextprotocol.io/docs/getting-started/intro`, 2024.

[18] M. Alla. Scalable mcp server-client architecture with fastmcp in microservices. *Sarcouncil Journal of Multidisciplinary*, 5:823–829, 2025.

[19] Anthropic. Introducing the model context protocol. `https://www.anthropic.com/news/model-context-protocol`, 2024.

[20] Google APIs. Introduction — mcp toolbox for databases. `https://googleapis.github.io/genai-toolbox/getting-started/introduction/`, 2025.

[21] Microsoft Corporation. Introducing model context protocol (mcp) in copilot studio: Simplified integration with ai apps and agents. `https://www.microsoft.com/en-us/microsoft-copilot/blog/copilot-studio/introducing-model-context-protocol-mcp-in-copilot-studio-simplified-integration-with-ai-apps-an` 2025.

[22] OpenAI. Model context protocol (mcp) — openai agents sdk. `https://openai.github.io/openai-agents-python/mcp/`.

[23] Andrea Matarazzo and Riccardo Torlone. A survey on large language models with some insights on their capabilities and limitations, 2025.

[24] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey, 2025.

[25] Lu XIANG, Yang ZHAO, Yaping ZHANG, and Chengqing ZONG. A survey of large language models in discipline-specific research: Challenges, methods and opportunities, March 2025.

[26] Semrush. Top websites ranking for "ai" in the world. `https://www.semrush.com/website/top/global/ai/`, 2025. Accessed: 25-09-2025.

[27] Andres Contreras. A gemini report: How many people are using generative ai on a daily basis?, December 2024.

[28] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024.

[29] hir0rameel. Qwen-claude llm. `https://ollama.com/hir0rameel/qwen-claude:latest`. Ollama Model repository.

[30] Martin Juan José Bucher and Marco Martini. Fine-tuned'small'llms (still) significantly outperform zero-shot generative ai models in text classification. *arXiv preprint arXiv:2406.08660*, 2024.

[31] Sam Houliston, Ambroise Odonnat, Charles Arnal, and Vivien Cabannes. Provable benefits of in-tool learning for large language models. *arXiv preprint arXiv:2508.20755*, 2025.

[32] Model Context Protocol. Model context protocol — overview (protocol revision 2025-06-18). `https://modelcontextprotocol.io/specification/2025-06-18/basic?utm`, 2025.

[33] Xinyun Chen Dawn Song. Llm agents: Brief history and overview. Lecture slides, UC Berkeley, 2024.

[34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2021.

[35] Weikai Xu, Chengrui Huang, Shen Gao, and Shuo Shang. Llm-based agents for tool learning: A survey: W. xu et al. *Data Science and Engineering*, pages 1–31, 2025.

[36] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[38] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. *URL https://arxiv. org/abs/2305.16291*, 2023.

[39] Ziming Li, Huadong Zhang, Chao Peng, and Roshan Peiris. Exploring large language model-driven agents for environment-aware spatial interactions and conversations in virtual reality role-play scenarios. In *2025 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, pages 1–11. IEEE, 2025.

[40] SongTang SongTang, Kaiyong Zhao, Lei Wang, Yuliang Li, Xuebo Liu, Junyi Zou, Qiang Wang, and Xiaowen Chu. Unrealllm: Towards highly controllable and interactable 3d scene generation by llm-powered procedural content generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19417–19435, 2025.

[41] Jonathan Linowes. *Unity 2020 virtual reality projects: Learn VR development by building immersive applications and games with Unity 2019.4 and later versions*. Packt Publishing Ltd, 2020.

[42] Unity Technologies. Xr interaction toolkit manual. `https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.1/manual/index.html`, 2021.

[43] Joanna Bergström, Tor-Salve Dalsgaard, Jason Alexander, and Kasper Hornbæk. How to evaluate object selection and manipulation in vr? guidelines from 20 years of studies. In *proceedings of the 2021 CHI conference on human factors in computing systems*, pages 1–20, 2021.

[44] Andreas Urech, Pascal Valentin Meier, Stephan Gut, Pascal Duchene, and Oliver Christ. Mapping or no mapping: the influence of controller interaction design in an immersive virtual reality tutorial in two different age groups. *Multimodal Technologies and Interaction*, 8(7):59, 2024.

[45] Junlong Chen, Jens Grubert, and Per Ola Kristensson. Large language model-assisted speech and pointing benefits multiple 3d object selection in virtual reality. *arXiv preprint arXiv:2410.21091*, 2024.

[46] Naveen Krishnan. Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications. *arXiv preprint arXiv:2504.21030*, 2025.

[47] Descope. What is the model context protocol (mcp) and how it works. `https://www.descope.com/learn/post/mcp`, 2025.

[48] Model Context Protocol. Model context protocol documentation. `https://modelcontextprotocol.io/docs/learn/architecture`, 2025.

[49] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp). *arXiv preprint arXiv:2505.02279*, 2025.

[50] Agent Communication Protocol. Welcome — agent communication protocol. `https://agentcommunicationprotocol.dev/introduction/welcome`, 2025.

[51] Agent Communication Protocol. How does acp compare to other ai protocols? `https://agentcommunicationprotocol.dev/about/mcp-and-a2a`, 2025.

[52] A2A Protocol. What is a2a? `https://a2a-protocol.org/latest/topics/what-is-a2a/`, 2025.

[53] A2A Protocol. Core concepts and components in a2a. `https://a2a-protocol.org/latest/topics/key-concepts/#interaction-mechanisms`, 2025.

[54] Partha Pratim Ray. A review on agent-to-agent protocol: Concept, state-of-the-art, challenges and future directions. May 2025.

[55] Agent Network Protocol. Agent network protocol guide. `https://agent-network-protocol.com/guide/`, 2025.

[56] Xianzheng Ma, Brandon Smart, Yash Bhalgat, Shuai Chen, Xinghui Li, Jian Ding, Jindong Gu, Dave Zhenyu Chen, Songyou Peng, Jia-Wang Bian, et al. When llms step into the 3d world: A survey and meta-analysis of 3d tasks via multi-modal large language models. *arXiv preprint arXiv:2405.10255*, 2024.

[57] Unity Technologies. Unity ai — benefits. https://unity.com/products/ai#benefits, 2025.

[58] Junlong Chen, Jens Grubert, and Per Ola Kristensson. Analyzing multimodal interaction strategies for llm-assisted manipulation of 3d scenes. In *2025 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, pages 206–216. IEEE, 2025.

[59] Unity Technologies. Unity ai assistant package - getting started. https://docs.unity3d.com/Packages/com.unity.ai.assistant@1.0/manual/getting-started.html, 2025.

[60] Unity Technologies. Unity ai assistant package - run overview. https://docs.unity3d.com/Packages/com.unity.ai.assistant@1.0/manual/run-overview.html, 2025.

[61] Unity Technologies. Unity ai assistant package - use /code mode. https://docs.unity3d.com/Packages/com.unity.ai.assistant@1.0/manual/code-overview.html, 2025.

[62] Unity Technologies. Unity ai guiding principles. https://unity.com/legal/unityai-guiding-principles, 2025.

[63] Shuo Duan, Chunyu Liu, Tianhua Rong, Yixin Zhao, and Baoge Liu. Integrating ai in medical education: a comprehensive study of medical students' attitudes, concerns, and behavioral intentions. *BMC Medical Education*, 25(1):599, 2025.

[64] G Feretzakis, K Papaspyridis, A Gkoulalas-Divanis, and VS Verykios. Privacy-preserving techniques in generative ai and large language models: A narrative review. information, 15 (11), 697, 2024.

[65] Giada Pistilli and Bruna Trevelin. Can ai be consentful? *arXiv preprint arXiv:2507.01051*, 2025.

[66] Microsoft. Microsoft azure security fundamentals overview. https://learn.microsoft.com/en-us/azure/security/fundamentals/overview#azures-defense-in-depth-security-approach, 2025.

[67] Quan Wei, Chung-Yiu Yau, Hoi-To Wai, Yang Katie Zhao, Dongyeop Kang, Youngsuk Park, and Mingyi Hong. Roste: An efficient quantization-aware supervised fine-tuning approach for large language models. *arXiv preprint arXiv:2502.09003*, 2025.

[68] Pranshav Gajjar and Vijay K Shah. Oransight-2.0: Foundational llms for o-ran. *arXiv preprint arXiv:2503.05200*, 2025.

[69] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, et al. A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. *ACM Transactions on Intelligent Systems and Technology*, 2024.

[70] CoderGamester. mcp-unity. https://github.com/CoderGamester/mcp-unity, 2025.

[71] Ivan Murzak. Unity mcp - default mcp tools documentation. https://github.com/IvanMurzak/Unity-MCP/blob/main/docs/default-mcp-tools.md, 2025.

[72] notargs. Unitynaturalmcp. https://github.com/notargs/UnityNaturalMCP, 2025.

[73] akiojin. unity-mcp-server. https://github.com/akiojin/unity-mcp-server, 2025.

[74] Shutong Wu and Justin P Barnett. Mcp-unity: Protocol-driven framework for interactive 3d authoring. In *Proceedings of the SIGGRAPH Asia 2025 Technical Communications*, pages 1–4. 2025.

[75] Mohit Nayak, Jari Kangas, and Roope Raisamo. A study of nlp-based speech interfaces in medical virtual reality. *Multimodal Technologies and Interaction*, 9(6):50, 2025.

[76] Zhimin Wang, Maohang Rao, Shanghua Ye, Weitao Song, and Feng Lu. Towards spatial computing: recent advances in multimodal natural interaction for xr headsets. *arXiv preprint arXiv:2502.07598*, 2025.

[77] Alice Cai, Caine Ardayfio, AnhPhu Nguyen, Tica Lin, and Elena Glassman. Atomxr: Streamlined xr prototyping with natural language and immersive physical interaction. *arXiv preprint arXiv:2311.11238*, 2023.

[78] Raveekiat Singhaphandu and Warut Pannakkong. A review on enabling technologies of industrial virtual training systems. *International Journal of Knowledge and Systems Science (IJKSS)*, 15(1):1–33, 2024.

[79] Lee Beever, Serban Pop, and Nigel W John. Leveled vr: A virtual reality level editor and workflow for virtual reality level design. In *2020 IEEE Conference on Games (CoG)*, pages 136–143. IEEE, 2020.

[80] Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, et al. Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios. *arXiv preprint arXiv:2401.17167*, 2024.

[81] jonigl. ollama-mcp-bridge. https://github.com/jonigl/ollama-mcp-bridge, 2025.

[82] AiKodex. Unity scene garage props. https://assetstore.unity.com/packages/3d/props/interior/simple-garage-197251, 2021.

[83] Gest. Unity scene furniture props. https://assetstore.unity.com/packages/3d/props/furniture/pack-gesta-furniture-1-28237, 2017.

[84] CoplayDev. unity-mcp: Model context protocol integration for unity. https://github.com/CoplayDev/unity-mcp, 2025.

[85] Chris Nolet. Quick outline. https://assetstore.unity.com/packages/tools/particles-effects/quick-outline-115488, 2022.

[86] Unity Technologies. Unity platform roadmap — editor. https://unity.com/roadmap/detail#unity-platform-editor, 2026.

[87] Clarky. Play mode saver. https://assetstore.unity.com/packages/tools/utilities/play-mode-saver-104836, 2021.

[88] shmily_006. shmily_006/Qw3:latest. `https://ollama.com/shmily_006/Qw3:latest`. Ollama model repository.

[89] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023.

[90] Jian Gong, Youwei Huang, Bo Yuan, Ming Zhu, Zhou Liao, Jianhang Liang, Juncheng Zhan, Jinke Wang, Hang Shu, Mingyue Xiong, et al. Ghostshell: Streaming llm function calls for concurrent embodied programming. *arXiv preprint arXiv:2508.05298*, 2025.

[91] Yanfei Zhang. Agent-as-tool: A study on the hierarchical decision making with reinforcement learning. *arXiv preprint arXiv:2507.01489*, 2025.

Appendix

A. Evaluation Test Protocol

The following list is the complete evaluation test protocol prompt list. This list defines the ordered sequence of natural language commands used during evaluation assessment in section 5.2.

1. *Create a cube named Hades.*
2. *Scale it <Hades> down.*
3. *Change the color of this <Hades> object to red.*
4. *Scale this <Sofa> up.*
5. *Disable its <Sofa> gravity.*
6. *Delete this <Sofa>.*
7. *Create a cube named Ares.*
8. *Move it <Ares> next to Hades.*
9. *Delete this <Hades>.*
10. *Create a small cube on top of the table.*
11. *Rename it <Small Cube> to Box.*
12. *Duplicate it <Box>.*
13. *Scale this <Box_Clone> up*
14. *Create a capsule named Poseidon.*
15. *Paint it <Poseidon> blue.*
16. *Scale this <Poseidon> up.*
17. *Create a cube next to this <Poseidon> game object.*
18. *Rename it <Newly Created Cube> to Artemis.*
19. *Change its <Artemis> color to orange.*
20. *Delete this <Poseidon>.*
21. *Duplicate this <Artemis>.*
22. *Turn off its <Artemis_Clone> gravity.*
23. *Move the printer next to the suitcase.*
24. *Create a green light source and name it Desk Light.*
25. *Move it <Desk Light> on top of the locker.*
26. *Increase its <Desk Light> brightness.*
27. *Create a capsule named Heracles.*
28. *Increase its <Heracles> size.*
29. *Disable this <Heracles> game object's gravity.*
30. *Paint it <Heracles> black.*

B. List of Tested Local LLMs

This is the list of all local LLMs that were tested during the development of the system. These models were checked to see whether they could generate structured MCP tool calls and interact reliably with the MCP server. Models that did not support tool usage or produced unreliable behavior were not included in the detailed evaluation presented in section 5.2.4. The list is not ranked in any particular order.

1. qwen3:8b
2. qwen3:14b
3. qwen3:32b
4. qwen3-next:80b
5. qwen3-coder:30b
6. shmily_006/Qw3:latest
7. hir0rameel/qwen-claude:latest
8. cnshenyang/qwen3-nothink:30b
9. dengcao/Qwen3-Reranker-8B:Q5_K_M
10. SimonPu/Qwen3-Coder:30B-Instruct_Q4_K_XL
11. rockn/DeepSeek-R1-0528-Qwen3-8B-IQ4_NL:latest
12. danielsheep/Qwen3-Coder-30B-A3B-Instruct-1M-Unsloth:latest
13. qwen2.5:3b
14. qwen2.5:14b
15. hhao/qwen2.5-coder-tools:3b
16. llama2:13b
17. llama3.1:8b
18. llama3.2:3b
19. llama3:70b
20. llama3-chatqa:8b
21. llama3.3:70b
22. ministral-3:8b
23. ministral-3:14b
24. gpt-oss:20b
25. gpt-oss:120b
26. glm4:9b
27. rnj-1:latest
28. devstral-small-2:24b
29. devstral-small-2:24b
30. firefunction-v2:70b

31. osmosis-ai/osmosis-mcp-4b
32. Mungert/osmosis-mcp-4b-GGUF:Q4_K_M
33. mradermacher/osmosis-mcp-4b-GGUF:Q4_K_M
34. mradermacher/MiniCPM4-MCP-i1-GGUF:Q4_K_M
35. mradermacher/qwen3_sft_playwright-GGUF:Q6_K
36. mradermacher/osmosis-mcp-4b-i1-GGUF:Q4_K_M
37. DevQuasar/openbmb.MiniCPM4-MCP-GGUF:Q4_K_M
38. tensorblock/openbmb_MiniCPM4-MCP-GGUF:Q4_K_M

Used Aids

Since English is not the native language of the author, during the preparation of this work, the author used DeepL Translate, ChatGPT and QuillBot in order to: grammar and spelling check, improve the academic tone, and literature scan. After using this tools, the author reviewed and edited the content as needed and take full responsibility for the thesis's content.

Date: 22/02/2026 Signature 

Statutory Declaration

I declare that I have authored this work independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Date: 22/02/2026 Signature